WISCH: Efficient data signing via correlated signatures

Ariel Futoransky[1], Ramses Fernandez[1], Emilio Garcia[1], Gabriel Larotonda[2,3], and Sergio Demian Lerner[1,4]

[1] Fairgate Labs
{futo, ramses.fernandez, emilio.garcia, sergio}@fairgate.io
[2] Universidad de Buenos Aires, Argentina
[3] CONICET, Argentina
glaroton@dm.uba.ar
[4] Rootstock Labs

Abstract. We present WISCH, a commit-reveal protocol that combines compact aggregate signatures with hash-based commitments to enable selective disclosure of correlated data in multiparty computation. The protocol separates an on-chain verification core from off-chain preparation, so that verification cost depends only on the number of openings, not on the size of the underlying message space. This yields asymptotic efficiency: on-chain cost grows linearly in the number of revealed items and is independent of the ambient domain, while the per-byte overhead decreases with the message granularity. Security is established via a simulation-based proof in a UC framework with an ideal ledger functionality, in the algebraic group and global random-oracle models, under standard assumptions for discrete-log-based signatures and hash-based commitments. Thus, WISCH provides selectively verifiable revelation with succinct on-chain checks and provable security guarantees.

Keywords: Winternitz signatures · Schnorr signatures · Multi-party computation.

1 Introduction and related work

We introduce WISCH, a protocol for handling correlated data in multiparty computation through encapsulated signature schemes, with a focus on enhancing efficiency and security in *blockchain-based fraud proofs*. The setup involves two primary parties, a prover $\mathcal{P}$ and a verifier $\mathcal{V}$, who agree on a function to evaluate. A trusted third party, such as a blockchain (modelled by an ideal ledger functionality), ensures the integrity of the process via timeouts and dispute resolution.

Although we model the underlying ledger as an abstract ideal functionality $\mathcal{F}_{\mathsf{B}}$, our concrete target is deployment on conservative UTXO-style blockchains such as Bitcoin with Taproot and Schnorr signatures. In such systems, the scripting language is intentionally limited: scripts are non-Turing complete, have

bounded size and stack usage, and offer only a small set of primitives (essentially: hash functions, integer operations, and verification of Schnorr signatures under a fixed group). In particular, there is no native support for bilinear pairings, generic large-field arithmetic, or precompiles for SNARK and polynomial-commitment verification. Any on-chain verifier must therefore be expressible as a short script that evaluates a handful of hashes and Schnorr verification equations. This is the main reason we focus on constructions whose verification core consists solely of these operations: WISCH is designed so that its on-chain checks can be implemented directly in this scripting model, whereas many state-of-the-art polynomial or vector commitments, and general-purpose ZK proof systems, cannot be verified efficiently (or at all) in this environment.

WISCH achieves a 6–9× reduction in on-chain storage compared to the Merkle + WOTS baseline used in BitVM-style protocols, representing approximately a 90% reduction in verification costs in the concrete parameter ranges we consider. The key innovation of WISCH is using *jointly created Schnorr signatures* to encapsulate (encrypt) WOTS signatures, providing both efficiency and security. The protocol begins with the prover generating WOTS key pairs and publishing the public keys. The parties then jointly create Schnorr signatures via a MuSig2-style multi-signature protocol [NRS21] to encapsulate WOTS signatures for all possible inputs, supporting efficient batch verification and selective revelation, augmented by zero-knowledge proofs of correct encapsulation. After a timeout, the prover reveals the relevant inputs by submitting selected Schnorr signatures, implicitly disclosing the chosen messages via the nonce–message binding.

WISCH builds upon foundational work in secure multiparty computation [Y82,Y86,GMW87,BGW88] and recent advances in signature aggregation protocols [NRS21,BDPT25]. Unlike generic MPC protocols, which typically optimise communication rounds or computational complexity, WISCH is explicitly designed to optimise *commitment size* and *on-chain verification cost* under conservative ledger constraints. Recent systems such as BitVM and its variants [AAL24,LAMJC24] utilise WOTS for disputes on Bitcoin-like ledgers; WISCH refines this approach via a two-layer hybrid construction that replaces Merkle-based openings with Schnorr-based encapsulation. To the best of our knowledge, no prior work has explored using one signature scheme to encrypt another while maintaining verifiability properties and compatibility with such ledgers.

The development of WISCH builds upon the research done in secure multiparty computation, recent advances in signature aggregation, and emerging verification systems. We briefly review the relevant prior work, identify the gaps our protocol addresses, and position WISCH within this landscape.

The foundational work of Yao [Y82,Y86] introduced garbled circuits for secure two-party computation, achieving constant-round complexity suitable for high-latency networks. The GMW protocol [GMW87] and the BGW protocol [BGW88] established general feasibility for multiparty scenarios, with the GMW completeness theorem showing that any polynomial-time function can be securely computed against computationally bounded adversaries. The BGW pro-

tocol achieves information-theoretic security with optimal corruption thresholds, while protocols such as SPDZ [DPSZ12] trade information-theoretic guarantees for practical efficiency using somewhat homomorphic encryption.

However, these generic MPC protocols are not optimised for scenarios where *commitment size* and *on-chain verification efficiency* are the primary constraints. They typically assume an interactive environment with online parties, whereas our setting requires non-interactive, publicly verifiable dispute resolution on a ledger. Threshold variants of MPC sometimes employ signatures to detect faults, which conceptually inspires our use of zero-knowledge proofs to certify correct encapsulation. The fair two-party protocols of Andrychowicz et al. [ADMM14], based on deposits and hash timeouts, also motivate our timeout-based design, though we replace simple hash commitments with encapsulated WOTS signatures to achieve better efficiency in our target setting. BitVM and BitVMX [AAL24,LAMJC24] utilise WOTS for MPC-style disputes in distributed ledgers; WISCH refines this hybrid approach, reducing storage needs by roughly 6–9× through Schnorr-based checks.

Commitment schemes are fundamental building blocks in MPC and fraud-proof protocols. Hash-based commitments using Merkle trees [M87] provide post-quantum security and efficient verification with $O(\log n)$ proofs. The development of hash-based signature schemes, such as Lamport's one-time signatures, Winternitz signatures, and modern constructions like SPHINCS+ [BHH+19], exhibits a trajectory towards more efficient post-quantum secure schemes. Winternitz signatures offer a tunable trade-off between signature size and computation time. WISCH employs the original WOTS scheme to construct a batch array of signatures (see Section B.1): each entry $W(m,k)$ signs a message $m \in \{0,1\}^N$ using a private key $\mathsf{priv}(k)$, with public keys of the form

$$\mathsf{pub}(k) = H^{15}_{\mathsf{win}}(\mathsf{priv}(k)) \quad \text{for } w = 4,$$

instantiated with a standard hash such as SHA-256 in the random-oracle model for simplicity and compatibility. Hülsing's W-OTS+ [H13] improves security margins at the cost of additional complexity; our analysis applies to such variants as well.

Discrete logarithm-based commitment schemes such as Pedersen commitments enable homomorphic operations that are useful in many MPC protocols, but they often require algebraic operations (e.g., pairings or structured reference strings) that are not available, or are very costly, on conservative ledgers. Recent advances in Schnorr signature aggregation, in particular MuSig2 [NRS21], enable multiple parties to collaboratively produce single signatures that are indistinguishable from individual Schnorr signatures. MuSig2's two-round signing process substantially reduces interaction while maintaining security against concurrent signing sessions. For joint signature generation and key derivation, WISCH modifies the MuSig2 flow to produce $2^N K$ joint signatures $(R(m,k), s(m,k))$ in two rounds: offline nonce exchanges yield aggregated nonces $R(m,k) = R_0(m,k) \cdot R_1(m,k) \cdot \eta(m,k)$, followed by partial signatures $s_i(m,k)$ that are combined asymmetrically (e.g., $\mathcal{V}$ sends its share to $\mathcal{P}$). Baum

et al. [BDPT25] establish UC security for interactive multi-signature schemes, providing the ideal functionality $\mathcal{F}_{\mathsf{IMS}}$ that we use in our hybrid model.

While hash-based and discrete-log-based commitments are both well studied on their own, prior work systematically combining their complementary properties in a way tailored to on-chain selective opening appears scarce, and we are not aware of direct applications like ours.

The idea of combining different cryptographic primitives to obtain enhanced properties has precedent in switch commitments [DN02], which alternate between perfectly hiding and perfectly binding regimes, and in hybrid encryption schemes that combine symmetric and asymmetric cryptography. Signature-based commitments using Camenisch–Lysyanskaya signatures [CL04] further demonstrate the value of using signatures as commitment schemes in UC frameworks.

The security analysis of such hybrids has advanced significantly with the development of the universal composability framework [C20] and the global random-oracle (GRO) model [CJS14]. Recent breakthroughs in UC security for succinct arguments [CF24] demonstrate that strong composability guarantees need not sacrifice efficiency. Bellare et al. [BDH20] study hybrids for read-only indifferentiability in the ROM, promoting modular composition; WISCH leverages this perspective to achieve IND-CPA secure encapsulation under the GRO, with adversarial advantage bounded by $\varepsilon_{\mathsf{ENC}}(\lambda)$. Fiat–Shamir style hybrids, such as combining Dilithium and Schnorr, are studied in [DFP23] and inform our EUF-CMA bounds for the outer Schnorr layer. The literature also considers concatenations of EC and PQ signatures, which resemble WISCH's nesting but without derivation-based encapsulation; we extend this line by employing HKDF-based key extraction from joint signatures to support selective decryption. Lightweight EC-based schemes such as LightDSA achieve signature sizes comparable to our 68 bytes per Schnorr check, which WISCH further amortises through batch linear combinations. Critiques of the algebraic group model (AGM) [FKL18] shape our algebraic-adversary definitions and our use of AOMDL-type assumptions.

To the best of our knowledge, no prior work has explored using one signature scheme to *encrypt* another while maintaining verifiability properties and compatibility with conservative ledgers. The specific combination of jointly created multi-signatures as encryption keys for deterministic one-time signatures represents a novel construction with a distinctive blend of security and efficiency.

Our main contributions are:

- A two-layer signature architecture in which Schnorr multi-signatures encapsulate WOTS signatures, enabling batch verification while preserving selective revelation capabilities.
- Concrete efficiency gains: a data-expansion factor of roughly 35:1 compared to about 200:1 for standard WOTS-based approaches, making previously impractical applications feasible in our target setting.
- A complete UC security proof showing that WISCH realizes an ideal commitment functionality $\mathcal{F}_{\mathsf{WISCH}}$ in the $(\mathcal{F}_{\mathsf{IMS}}, \mathcal{F}_{\mathsf{B}})$-hybrid model under standard assumptions in the AGM and GRO models.

– Practical compatibility with existing commitment frameworks and ledger-based verification services, relying only on primitives (Schnorr signatures and hash-based signatures) that are natively supported or easily emulated on conservative blockchains.

2 Preliminaries

2.1 Notation and conventions

Throughout this paper, we denote the concatenation of binary strings a and b as $a \parallel b$. Messages are represented as bit-strings of length N, that is, $m \in \{0,1\}^N$. We fix $K \in \mathbb{N}$ as the number of protocol slots/columns (a.k.a. challenge rounds). Indices are $k \in \{0,\dots,K-1\}$. Unless stated otherwise, N and w denote the message bitlength and the Winternitz radix, respectively. We write $a = b \pmod p$ to denote congruence modulo p, and use $\mathbb{Z}_p^*$ to represent the group of invertible elements modulo p. The security parameter is λ. We use n for the number of signers in multi-signatures. Hash outputs bitlengths use κ, and κ_{KDF} for KDF key bitlength. The Schnorr scalar bitlength is ℓ_s.

The protocol participants are denoted as $\mathcal{P}$ for Patrick (the prover) and $\mathcal{V}$ for Valery (the verifier), with their corrupt versions indicated by $\mathcal{P}^\star$ and $\mathcal{V}^\star$ respectively. All cryptographic operations take place over an elliptic curve E with generator g, operating in the field $\mathbb{Z}_p$ where p is the curve's prime order. For points $R = (x_R, y_R)$ on the elliptic curve, we write $R \parallel m$ for the concatenation of x_R and m in their binary representations. All hash functions are defined $H : \{0,1\}^* \to \mathbb{Z}_p$, unless otherwise specified. Each protocol instance carries a session identifier *sid*, included as associated data in encryptions and as a domain-separation tag in hashes.

We treat λ as the cryptographic security parameter. The symbol width N is an efficiency knob independent of λ; throughout we assume $2^N \le \mathrm{poly}(\lambda)$. In our concrete settings we fix $N \in \{4, 8\}$ (so $2^N \in \{16, 256\}$), and the payload scales by increasing K (total bits $= N \cdot K$), not by increasing N. Under this regime, all algorithms—including the simulators—run in time $\mathrm{poly}(\lambda, K)$.

2.2 Cryptographic primitives

Winternitz one-time signatures Let λ be the security parameter, w the Winternitz radix, and $H_{\mathsf{win}} : \{0,1\}^\star \to \{0,1\}^\kappa$ a hash. For a message bitlength N, let $M = \lceil N/w \rceil$, $C = \lfloor \log_2(M(2^w - 1))/w \rfloor + 1$, and $\ell = M + C$. Write $b(m) = (b_0(m), \dots, b_{\ell-1}(m)) \in \{0, \dots, 2^w - 1\}^\ell$ for the base-2^w digits of m concatenated with its checksum.

– KeyGen: sample $\mathsf{priv}(k) = (\mathsf{priv}_0(k), \dots, \mathsf{priv}_{\ell-1}(k))$ with $\mathsf{priv}_i(k) \xleftarrow{\$} \{0,1\}^\kappa$ and set $\mathsf{pub}_i(k) = H_{\mathsf{win}}^{2^w-1}\big(\mathsf{priv}_i(k)\big)$ for $i = 0, \dots, \ell - 1$. Let us denote $\mathsf{pub}(k) = (\mathsf{pub}_0(k), \dots, \mathsf{pub}_{\ell-1}(k))$.

– Sign: on input $m \in \{0,1\}^N$, output

$$W(m,k) = \big(H_{\mathsf{win}}^{b_0(m)}(\mathsf{priv}_0(k)), \ldots, H_{\mathsf{win}}^{b_{\ell-1}(m)}(\mathsf{priv}_{\ell-1}(k))\big).$$

– Verify: given $(m, W(m,k), \mathsf{pub}(k))$, accept iff for all i,

$$H_{\mathsf{win}}^{2^w-1-b_i(m)}\big(W_i(m,k)\big) = \mathsf{pub}_i(k).$$

For each $k \in \{0, \ldots, K-1\}$ we obtain a WOTS key pair $(\mathsf{priv}(k), \mathsf{pub}(k))$ via KeyGen. Let $W(m,k)$ denote the WOTS signature of m under $(\mathsf{priv}(k), \mathsf{pub}(k))$. We can view all $2^N \times K$ signatures as a matrix whose k-th column is given by $\{W(m,k)\}_{m\in\{0,1\}^N}$; in the protocol only the entry $W(m(k),k)$ is ever opened per column.

The signature of m with a private key $\mathsf{priv}(k) = (\mathsf{priv}_0(k), \ldots, \mathsf{priv}_{\ell-1}(k))$ is: $W(m,k) = (\mathrm{H}_{\mathsf{win}}{}^{b_0(m)}(\mathsf{priv}_0(k)), \ldots, \mathrm{H}_{\mathsf{win}}{}^{b_{\ell-1}(m)}(\mathsf{priv}_{\ell-1}(k)))$ where $b_i(m)$ are the base-2^w digits of m and its checksum. We define the public keys as $\mathsf{pub}_i(k) = \mathrm{H}_{\mathsf{win}}{}^{2^w-1}(\mathsf{priv}_i(k))$.

The MuSig2 protocol MuSig2 [NRS21] enables the creation of joint Schnorr signatures. Let $x_P, x_V \in \mathbb{Z}_p$ be the signers' secret keys for $X_P = g^{x_P}$, $X_V = g^{x_V}$ their public keys on E, and define $L = \mathrm{H}_{\mathsf{agg}}(X_P \parallel X_V)$, and the coefficient $a_i = \mathrm{H}_{\mathsf{agg}}(L \parallel i \parallel X_i)$ $(i \in \{P,V\})$. The session's aggregated public key is

$$X := X_P^{a_P} X_V^{a_V}.$$

This X is fixed for the session *sid* and is used in all (m,k) signatures. The joint signature is $(R,s) = (R_0 R_1^{\eta}, s_P + s_V \pmod p)$, for $\eta = \mathrm{H}_{\mathsf{non}}(X \parallel R_0 \parallel R_1 \parallel m)$.

For each (m,k), the challenge is $c(m,k) = \mathrm{H}_{\mathsf{sig}}(X \parallel R(m,k) \parallel m)$ and the joint scalar is $s(m,k) = s_P(m,k) + s_V(m,k) \pmod p$, and the checking $g^{s(m,k)} = R(m,k)X^{c(m,k)}$ is performed for correctness.

MuSig2 is an interactive two-round signing protocol. In each session with aggregated key X and message m:

1. **Round 1 (nonce exchange)**: Each signer $i \in \{P,V\}$ samples a fresh nonce r_i, sends $R_i = g^{r_i}$, and both compute the aggregate $R = R_P \cdot R_V$.
2. **Round 2 (signature shares)**: Each signer computes the challenge $c = H_{\mathsf{sig}}(X \parallel R \parallel m)$ and a share $s_i = r_i + a_i x_i \cdot c \pmod q$ (with a_i the key-agg coefficient). The final signature is (R,s) with $s = s_P + s_V \pmod q$.

We allow many concurrent sessions; security then refers to the UC-secure ideal functionality $\mathcal{F}_{\mathsf{IMS}}$ we cite for MuSig2, and our hybrids replace real MuSig2 calls with $\mathcal{F}_{\mathsf{IMS}}$.

For batching, define:

$$\begin{aligned} h_s(m,k) &= \mathrm{H}_{\mathsf{sig}}(s(m,k)), \\ H(m) &= \mathrm{H}_{\mathsf{sig}}(s(m,0) \parallel \cdots \parallel s(m,K-1)) \\ l(m,k) &= \mathrm{H}_{\mathsf{sig}}(H(m) \parallel k) \\ s'(m) &= \sum_k l(m,k)s(m,k) \end{aligned}$$

Then we check $g^{s'(m)} \stackrel{?}{=} \prod_k X^{c(m,k)l(m,k)} \prod_k R(m,k)^{l(m,k)}$.

Remark 1 (Domain separation). Throughout, we use domain-separated hash functions $\mathrm{H}_{\mathsf{agg}}, \mathrm{H}_{\mathsf{non}}, \mathrm{H}_{\mathsf{sig}}, \mathrm{H}_{\mathsf{win}}$, and HKDF. In the global random oracle model, these are instantiated via a single oracle $\mathcal{G}_{\mathsf{RO}}$ with distinct tags:

$$\begin{aligned}
\mathrm{H}_{\mathsf{agg}}(x) &:= \mathcal{G}_{\mathsf{RO}}(\text{“}\mathsf{agg}\text{”}\|x),\\
\mathrm{H}_{\mathsf{non}}(x) &:= \mathcal{G}_{\mathsf{RO}}(\text{“}\mathsf{non}\text{”}\|x),\\
\mathrm{H}_{\mathsf{sig}}(x) &:= \mathcal{G}_{\mathsf{RO}}(\text{“}\mathsf{sig}\text{”}\|x),\\
\mathrm{H}_{\mathsf{win}}(x) &:= \mathcal{G}_{\mathsf{RO}}(\text{“}\mathsf{win}\text{”}\|x),\\
\mathsf{HKDF}(s,y) &:= \mathcal{G}_{\mathsf{RO}}(\text{“}\mathsf{kdf}\text{”}\|s\|y).
\end{aligned}$$

This ensures that queries to different oracles are independent, which is essential for the security analysis.

2.3 Security assumptions

Zero-knowledge in the GRO model and straight-line extraction

Definition 1 (Zero-knowledge in GRO). *A proof system Π for relation R is $\epsilon_{\mathsf{ZK}}(\lambda)$-zero-knowledge in the global random-oracle (GRO) model if there exists a PPT simulator S such that for every PPT distinguisher D and auxiliary input z, when both experiments use the same GRO instance,*

$$\left|\Pr[D^{\mathsf{GRO}}(\{\Pi.\mathsf{Prove}(x_i,w_i)\}_{i=1}^{q},z)=1]-\Pr[D^{\mathsf{GRO}}(\{S(x_i)\}_{i=1}^{q},z)=1]\right| \le \epsilon_{\mathsf{ZK}}(\lambda),$$

for any adaptively chosen $(x_i,w_i)\in R$ with $q=\mathrm{poly}(\lambda)$.

Definition 2 (Simulation soundness in the GRO model). *Let us consider $R\subseteq\{0,1\}^\star\times\{0,1\}^\star$ an NP relation and let $\Pi=(\mathsf{Prove},\mathsf{Verify},\mathsf{Sim})$ be a non-interactive zero-knowledge proof system with simulator Sim. We say Π is $\epsilon_{\mathsf{SS}}(\lambda)$-simulation sound in the global random oracle model if for any PPT adversary $\mathcal{A}$:*

$$\Pr\begin{bmatrix}\mathsf{Verify}(x^\star,\pi^\star)=1\\ \wedge\ x^\star\notin\mathcal{L}_R\\ \wedge\ (x^\star,\pi^\star)\notin Q\end{bmatrix}\le\epsilon_{\mathsf{SS}}(\lambda)$$

where the experiment proceeds as follows:

1. *$\mathcal{A}$ has access to the global random oracle $\mathcal{G}_{\mathsf{RO}}$.*
2. *$\mathcal{A}$ has access to a simulation oracle $\mathcal{O}_{\mathsf{Sim}}$ that, on input a* true *statement $x\in\mathcal{L}_R$, returns $\pi\leftarrow\mathsf{Sim}(x)$ and records (x,π) in Q.*
3. *$\mathcal{A}$ outputs $(x^\star,\pi^\star)$.*

Here $\mathcal{L}_R=\{x:\exists w,(x,w)\in R\}$ is the language defined by R, and Q is the set of query-response pairs from $\mathcal{O}_{\mathsf{Sim}}$.

Informally: even after seeing polynomially many simulated proofs for true statements (produced without witnesses), $\mathcal{A}$ cannot produce an accepting proof for a false statement.

Definition 3 (Straight-line knowledge extraction). *Let $R \subseteq \mathcal{X} \times \mathcal{W}$ and Π be a proof system for R. Π has $\epsilon_{\mathsf{SLKE}}(\lambda)$ straight-line knowledge extraction if there exists a PPT extractor $\mathfrak{E}$ such that for any PPT prover $\mathcal{P}^\star$ and any $x \in \mathcal{X}$:*

1. *Straight-line: $\mathfrak{E}$ runs in expected polynomial time and interacts with $\mathcal{P}^\star$ without rewinding.*
2. *GRO interface: $\mathfrak{E}$ shares access to the GRO: it observes $\mathcal{P}^\star$'s queries and may program fresh inputs (with domain separation).*
3. *Extraction: Whenever $\mathcal{P}^\star$ makes the verifier accept on x, $\mathfrak{E}^{\mathcal{P}^\star}(x)$ outputs w with $(x, w) \in R$ except with probability $\epsilon_{\mathsf{SLKE}}(\lambda)$.*

Both properties hold for standard Σ-protocols transformed via Fischlin in the GRO with domain-separated oracles; this yields SLKE and avoids rewinding under concurrency.

The security of WISCH relies on several assumptions. For the MuSig2 component, we require existential unforgeability under chosen message attacks, which ensures that no polynomial-time adversary can forge valid multi-signatures with advantage exceeding $\epsilon_{\mathsf{IMS}}(\lambda)$. The Winternitz signatures provide existential unforgeability under one-time chosen message attacks, guaranteeing that after seeing one signature, an adversary cannot produce a valid signature for a different message except with probability at most $\epsilon_{\mathsf{W}}(\lambda)$.

Our zero-knowledge proof system satisfies straight-line knowledge extraction in the global random oracle model, meaning there exists an extractor that can obtain the witness from any accepting proof without rewinding, failing with probability at most $\epsilon_{\mathsf{SLKE}}(\lambda)$. The encryption scheme used to encapsulate Winternitz signatures achieves indistinguishability under chosen plaintext attacks, ensuring that ciphertexts reveal no information about their underlying plaintexts with security bound $\epsilon_{\mathsf{ENC}}(\lambda)$.

The trusted third party functionality $\mathcal{F}_{\mathsf{B}}$, which may be instantiated as a blockchain or similar verification service, provides both integrity and liveness guarantees. The integrity property ensures that commitments resolve correctly according to their verification conditions with failure probability at most ϵ_I, while the liveness property guarantees that all commitments eventually reach resolution within the specified time bounds with failure probability at most ϵ_L.

We work in the Algebraic Group Model (AGM) and assume the One-More Discrete Log for the multi-signature component; hash functions and HKDF are modelled within a Global Random Oracle. These assumptions collectively ensure that the protocol achieves its security goals under composition, as formalized in our main theorems. Formal definitions of these security properties appear in Appendix C.

3 The WISCH Protocol

3.1 Protocol overview

WISCH operates in three main phases with timeouts $t_{KO} < t_0 < t_1$ where all timing is measured by $\mathcal{F}_{\mathsf{B}}$'s monotonic clock:

1. Setup: the prover $\mathcal{P}$ generates K Winternitz key pairs and publishes public keys. $\mathcal{P}$ and $\mathcal{V}$ jointly create $2^N \times K$ Schnorr signatures using MuSig2, where each signature corresponds to a possible input-step pair (m, k). $\mathcal{P}$ encrypts his Winternitz signatures using authenticated encryption with associated data (AEAD): $E(m,k) = \mathsf{Enc}^{\mathrm{ad}(sid,k,\mathsf{pub}(k))}_{k_{\mathsf{enc}}(m,k)}(W(m,k))$ where the key $k_{\mathsf{enc}}(m,k) = \mathsf{HKDF}(s(m,k), m \parallel k)$ and $\mathrm{ad}(sid, k, \mathsf{pub}(k))$ is associated data binding each ciphertext to its session and public key. $\mathcal{P}$ provides a zero-knowledge proof of correct encapsulation. We bind each ciphertext to its public context using associated data $\mathrm{ad}(sid, k, \mathsf{pub}(k)) := \langle sid \rangle \parallel \langle k \rangle \parallel \langle \mathsf{pub}(k) \rangle$. Hashes $\mathrm{H}_{\mathsf{sig}}, \mathrm{H}_{\mathsf{non}}$ and HKDF are treated as disjoint domains of a global random oracle via domain-separation tags.
2. Commitment: after time t_{KO}, $\mathcal{P}$ reveals his chosen inputs $\{m(k)\}_{k=0}^{K-1}$ by submitting the corresponding Schnorr signatures $(R(m(k),k), s(m(k),k))$ to the trusted party $\mathcal{F}_{\mathsf{B}}$.
3. Verification: $\mathcal{V}$ extracts inputs from the unique nonces, decrypts the Winternitz signatures, and verifies both their validity and that $V(\{m(k)\}) = 1$. She can challenge before timeout t_1 if verification fails.

3.2 Trusted third party functionality

The WISCH protocol relies on a trusted third party functionality $\mathcal{F}_{\mathsf{B}}$, which serves as an arbiter for commitment verification and dispute resolution. In practical deployments, this functionality can be instantiated by a blockchain smart contract, a trusted hardware module, or any system providing reliable state management and deterministic execution. The key role of $\mathcal{F}_{\mathsf{B}}$ is to maintain commitment integrity while ensuring liveness properties that prevent indefinite protocol suspension. Unlike traditional two-party protocols where disputes may remain unresolved, $\mathcal{F}_{\mathsf{B}}$ provides definitive resolution through a timeout mechanism, guaranteeing that either $\mathcal{P}$ successfully demonstrates input validity or $\mathcal{V}$ successfully challenges invalid claims within bounded time periods t_0 and t_1.

The functionality maintains two commitments: $\texttt{commit}_0$ containing K aggregated Schnorr checksigs with timeout t_0 favouring $\mathcal{V}$, and $\texttt{commit}_1$ containing Winternitz public keys and the verification predicate V with timeout t_1 favouring $\mathcal{P}$. This structure ensures that $\mathcal{P}$ must first prove possession of valid signatures (via $\texttt{commit}_0$) before claiming victory through timeout (via $\texttt{commit}_1$), while $\mathcal{V}$ retains the ability to challenge any invalid inputs before the final timeout. The functionality provides $(\epsilon_I, \epsilon_L, t)$-security, where ϵ_I bounds the probability of incorrect resolution and ϵ_L bounds the probability of liveness failure.

Algorithm 1 Trusted Commitment Service $\mathcal{F}_{\mathsf{B}}$ (Main components)

State: Commitments, timeouts, witnesses
procedure CreateCommitment(parties, timeout, winner)
    Generate commitment ID cid
    Store (parties, timeout, winner)
    return cid
end procedure
procedure SubmitWitness(party, cid, witness)
    Verify party $\in$ commitment.parties
    Store witness for verification
    Execute verification script if triggered
end procedure
procedure ResolveTimeout(cid)
    if current_time $\geq$ timeout **then**
        Resolve for timeout_winner
end procedure

3.3 Core protocol

The WISCH protocol achieves efficient commitment with selective revelation. At its core, the protocol leverages an asymmetric property: while creating commitments requires generating $2^N \times K$ signature pairs during setup, verification requires checking K signatures corresponding to the actually revealed inputs. This asymmetry is important for applications where preparation can be performed offline but verification must be efficient, such as blockchain-based dispute resolution or resource-constrained verification environments.

In the offline setup, the parties exchange nonces in two rounds, but at scale: $2 \times 2^N \times K$ nonces are combined to form the (R_0, R_1) pairs across all $(m, k) \in \{0,1\}^N \times [K]$. This up-front cost enables the later reveal to touch only the K opened entries.

The protocol has a nested signature structure. Each Winternitz signature $W(m, k)$ is encrypted using a key derived from the corresponding Schnorr signature $s(m, k)$, creating a two-layer commitment where the outer layer (Schnorr) protects the inner layer (Winternitz). The binding between layers occurs through the key derivation $k_{\mathsf{enc}}(m, k) = \mathsf{HKDF}(s(m, k), m \parallel k)$, ensuring that only the party possessing the complete Schnorr signature can decrypt the corresponding Winternitz signature. The zero-knowledge proof provided by $\mathcal{P}$ during setup confirms this structure without revealing the actual signatures, preventing $\mathcal{P}$ from later claiming different inputs than those originally committed. The protocol execution follows strict timing constraints with $t_{KO} < t_0 < t_1$, where t_{KO} initiates the revelation phase, t_0 bounds the commitment submission period, and t_1 provides the final challenge window.

Algorithm 2 WISCH Protocol (Main components)

Require: Parameters $K, N, w = 4$; timeouts $t_{KO} < t_0 < t_1$; verification predicate V; session identifier sid

Setup (offline):
for $k \in \{0, \dots, K-1\}$ **do**
  Generate WOTS key pair $(\mathsf{priv}(k), \mathsf{pub}(k))$ ▷ $(\mathsf{priv}(k), \mathsf{pub}(k)) \leftarrow \mathsf{WOTS.KeyGen}(1^\lambda)$
end for
for $(m, k) \in \{0,1\}^N \times [K]$ **do**
  Run MuSig2 precomputation for m at index k to get $(R(m,k), s(m,k))$
  Derive encryption key $k_{\mathsf{enc}}(m,k) \leftarrow \mathsf{HKDF}(s(m,k),\ sid \parallel k)$
  Compute deterministic WOTS signature $W(m,k) \leftarrow \mathsf{WOTS.Sign}(\mathsf{priv}(k), m)$
  Encrypt $E(m,k) \leftarrow \mathsf{Enc}^{\mathrm{ad}(sid,k,\mathsf{pub}(k))}_{k_{\mathsf{enc}}(m,k)}\big(W(m,k)\big)$
end for
$\mathcal{P}$ publishes the table $\{E(m,k)\}_{m,k}$ and a ZK proof of correct encapsulation
$\mathcal{P}$ creates two commitments with $\mathcal{F}_{\mathsf{B}}$ (cid_0 with timeout t_0, cid_1 with timeout t_1)
Commitment phase (after t_{KO}):
$\mathcal{P}$ chooses one message $m(k) \in \{0,1\}^N$ for each $k \in \{0, \dots, K-1\}$
for $k \in \{0, \dots, K-1\}$ **do**
  $\mathcal{P}$ submits $(R(m(k),k), s(m(k),k))$ to $\mathcal{F}_{\mathsf{B}}$ under commitment cid_0
end for
if by time t_0 some index k has no valid pair $(R(m(k),k), s(m(k),k))$ opened **then**
  $\mathcal{V}$ wins (timeout on the outer commitment)
end if
Verification phase:
for $k \in \{0, \dots, K-1\}$ **do**
  $\mathcal{V}$ obtains $(R(m(k),k), s(m(k),k))$ from $\mathcal{F}_{\mathsf{B}}$
  Using nonce–message injectivity, $\mathcal{V}$ recovers $m(k)$ as the unique $m \in \{0,1\}^N$ with matching $R(m,k)$
  Recompute $k_{\mathsf{enc}}(m(k),k) \leftarrow \mathsf{HKDF}(s(m(k),k),\ sid \parallel k)$
  Decrypt $W(m(k),k) \leftarrow \mathsf{Dec}^{\mathrm{ad}(sid,k,\mathsf{pub}(k))}_{k_{\mathsf{enc}}(m(k),k)}\big(E(m(k),k)\big)$
  Check $\mathsf{WOTS.Verify}\big(m(k), W(m(k),k), \mathsf{pub}(k)\big)$
end for
if any verification fails **or** $V(\{m(k)\}_{k=0}^{K-1}) = 0$ **then** $\mathcal{V}$ issues a challenge via $\mathcal{F}_{\mathsf{B}}$ before t_1
else $\mathcal{P}$ wins after t_1
end if

3.4 Batch verification optimization

To enable efficient verification, $\mathcal{P}$ sends commitments $\mathsf{hs}(m,k) = \mathrm{H}_{\mathsf{sig}}(s(m,k))$, and $H(m) = \mathrm{H}_{\mathsf{sig}}(s(m,0) \parallel \cdots \parallel s(m,K-1))$. Let $c(m,k) = \mathrm{H}_{\mathsf{sig}}(X \parallel R(m,k) \parallel m)$ denote the Schnorr challenge. $\mathcal{V}$ can batch-verify using linear combinations $s'(m) = \sum_k l(m,k)s(m,k)$ where $l(m,k) = \mathrm{H}_{\mathsf{sig}}(H(m) \parallel k)$:

$$g^{s'(m)} \stackrel{?}{=} \prod_{k=0}^{K-1} X^{c(m,k)l(m,k)} \prod_{k=0}^{K-1} R(m,k)^{l(m,k)}$$

This reduces verification from $2^N K$ individual checks to 2^N batch checks.

3.5 Zero-knowledge relation

The zero-knowledge proof sets that $\mathcal{P}$ has a witness satisfying the relation R_{WISCH}, for all m, k and i:

$$\begin{aligned}
\mathsf{hs}(m,k) &= \mathrm{H}_{\mathsf{sig}}(s(m,k)) \\
H(m) &= \mathrm{H}_{\mathsf{sig}}(s(m,0) \parallel \cdots \parallel s(m,K-1)) \\
s'(m) &= \sum_{k=0}^{K-1} l(m,k)s(m,k) \\
\mathsf{pub}_i(k) &= {\mathrm{H}_{\mathsf{win}}}^{2^w-1-b_i(m)}(w_i(m,k)) \\
E(m,k) &= \mathsf{Enc}^{\mathrm{ad}(sid,k,\mathsf{pub}(k))}_{k_{\mathsf{enc}}(m,k)}(W(m,k))
\end{aligned}$$

This ensures $\mathcal{P}$ commits to specific inputs and cannot later claim different values. (Complete protocol in Appendix B.)

Remark 2. The object $W(m,k)$ can be any deterministic one-time signature with a public Verify predicate; the ZK relation simply asserts correct encryption of a valid $W(m,k)$. Our proofs remain unchanged under this substitution.

4 Performance Analysis

WISCH achieves significant efficiency improvements over traditional hash-based signatures. The key insight is that Schnorr signatures require only 68 bytes per check when using the same public key, while WOTS requires hundreds of bytes per signature.

4.1 Storage comparison

For K signatures, WISCH requires:

$$\mathrm{Size}_{\mathsf{WISCH}} = 33 + K \times 68 \text{ bytes}$$

where 33 bytes store the aggregated public key X (amortized across all signatures) and each signature check requires 68 bytes.

For large K, WISCH uses approximately 68 bytes per signature versus 384-640 bytes for optimized WOTS, representing an 85-90% reduction in on-chain storage. This improvement makes previously impractical applications viable in resource-constrained environments like blockchains (detailed calculations in Appendix D).

Scheme	Parameters	Size (bytes)	Reduction
WISCH	Any K	$33 + K \times 68$	—
WOTS	$N = 16, w = 4$	$K \times 384$	5.6×
WOTS	$N = 24, w = 4$	$K \times 512$	7.5×
WOTS	$N = 32, w = 4$	$K \times 640$	9.4×
Lamport	$N = 24$	$K \times 2304$	47×

Table 1. On-chain storage requirements for K signatures

5 Security Analysis

We analyze WISCH's security in the universal composability framework with a global random oracle. We prove that the protocol achieves security against static corruption of either the prover or verifier (but not both simultaneously). In the appendices we tackle the mixed-role corruption model to show the single-role ideas apply with minimal assumptions.

5.1 Ideal functionality $\mathcal{F}_{\mathsf{WISCH}}$

We specify the commitment–reveal behaviour targeted by our protocol. This functionality $\mathcal{F}_{\mathsf{WISCH}}$ (Algorithm 3) interacts with a prover $\mathcal{P}$, a verifier $\mathcal{V}$, a simulator $\mathcal{S}$, and functionalities $\mathcal{F}_{\mathsf{B}}$ and $\mathcal{F}_{\mathsf{IMS}}$ [BDPT25, Section 3.4]. The functionality captures:

1. A two-commit timing policy via $\mathcal{F}_{\mathsf{B}}$ with timeouts $t_0 < t_1$ (and a kickoff t_{KO} if applicable).
2. Selective opening: after t_0, $\mathcal{P}$ may open any $S \subseteq [K]$; the functionality reveals the pairs $\{(k, m_k)\}_{k \in S}$ to $\mathcal{V}$.
3. Automatic accept/reject: accepts iff the opened entries are consistent with an abstract validity relation R; $\mathcal{V}$ may dispute before t_1. Only the opened indices/values and accept/reject bits leak.

Remark 3. In our instantiation, $R(m, k, w)$ asserts that w is a valid witness for index k and message m (for instance: a deterministic one-time signature under the k-th public key) and that the global predicate V holds over the set of opened pairs. The functionality never inspects w; existence is guaranteed by the Ideal/Real proof.

Remark 4. We assume static single-role corruption. The adversary controls delivery order and can drop messages subject to t_0, t_1. The functionality leaks only session identifiers, commit events, the sets S as they are revealed, the opened pairs (k, m_k), and the accept/reject bits.

Let $\mathbf{y} = (y_0, \ldots, y_{K-1}) \in \mathcal{Y}^K$ denote the per-index public data for this session and let $\hat{\mathbf{y}} := \mathsf{Com}(\mathbf{y}; r)$ be a binding commitment to $\mathbf{y}$. $\mathcal{F}_{\mathsf{WISCH}}$ records $\hat{\mathbf{y}}$ but

Algorithm 3 Functionality $\mathcal{F}_{\mathsf{WISCH}}$

Parameters: message universe $\mathcal{M}$, width K, timeouts $t_0 < t_1$
Relation: $R \subseteq \mathcal{M} \times [K] \times \mathcal{W}$
Parties: prover $\mathcal{P}$, verifier $\mathcal{V}$; adversary $\mathcal{A}$ controls delivery/timing
State: *state* $\in \{\texttt{init}, \texttt{committed}, \texttt{revealed}, \texttt{completed}\}$; hidden table $\{m_k\}_{k\in[K]}$ and witnesses $\{w_k\}_{k\in[K]}$ (never leaked)
procedure INIT
    On matching $(\mathsf{Init}, \mathsf{sid})$ from both parties, notify $\mathcal{A}$; set *state* $\leftarrow$ `init`
end procedure
procedure COMMIT_0
    Upon $(\mathsf{Commit}_0, \mathsf{sid})$ from $\mathcal{P}$ before t_0, record and leak event $(\mathsf{Commit}_0, \mathsf{sid})$ to $\mathcal{A}$
end procedure
procedure COMMIT_1($\hat{\mathbf{y}}$)
    Upon $(\mathsf{Commit}_1, \mathsf{sid}, \hat{\mathbf{y}})$ from $\mathcal{P}$ before t_1, record and leak event $(\mathsf{Commit}_1, \mathsf{sid})$ to $\mathcal{A}$
end procedure
procedure PROGRAMTABLE
    Once both commits exist (and before t_1), fix hidden $\{(m_k, w_k)\}_{k\in[K]}$ with $R(m_k, k, w_k)$ for all k
    If $\mathcal{P}$ is corrupted, $\mathcal{A}$ chooses (m_k, w_k); else the functionality chooses any consistent values
end procedure
procedure REVEAL($S \subseteq [K]$)
    After t_0, upon $(\mathsf{Reveal}, \mathsf{sid}, S)$ from $\mathcal{P}$, deliver $(\mathsf{Open}, \mathsf{sid}, \{(k, m_k)\}_{k\in S})$ to $\mathcal{V}$ and $\mathcal{A}$; mark S as opened
end procedure
procedure VERIFY
    Correctness/Binding/Validity guarantees: for all opened k, the opened m_k matches the hidden table; openings are binding; and $\exists w_k$ s.t. $R(m_k, k, w_k)$
    Output $(\mathsf{Accept}, \mathsf{sid}, S)$ to $\mathcal{V}$ when the above holds
end procedure
procedure DISPUTE(k, m')
    If $(\mathsf{Dispute}, \mathsf{sid}, k, m')$ from $\mathcal{V}$ before t_1 conflicts with any opening, output $(\mathsf{Reject}, \mathsf{sid}, k)$ to $\mathcal{V}$
end procedure
procedure LEAKAGE/CONTROL
    $\mathcal{A}$ learns session ids, commit events, each revealed set S, the opened (k, m_k) and final accept/reject bits; controls delivery order subject to t_0, t_1
end procedure

does not inspect its contents. In our concrete instantiation, $\mathbf{y}$ consists of the K deterministic-OTS (WOTS) verification keys (and fixed auxiliary parameters if any); hence $\hat{\mathbf{y}}$ can be implemented as a hash/Merkle root. Upon opening a set $S \subseteq [K]$, the prover reveals $\{y_k\}_{k \in S}$ with decommitments to $\hat{\mathbf{y}}$.

5.2 Main security theorem

We now formalize WISCH's security properties within the universal composability (UC) framework [C20], demonstrating that the protocol securely realizes an ideal commitment functionality even when composed with arbitrary other protocols. The UC framework provides the strongest known security guarantees for cryptographic protocols, ensuring that security properties are preserved under concurrent execution and protocol composition. Our analysis operates in a hybrid model where parties have access to both the multi-signature functionality $\mathcal{F}_{\mathsf{IMS}}$ (realized by MuSig2) and the trusted commitment service $\mathcal{F}_{\mathsf{B}}$, while sharing a global random oracle $\mathcal{G}_{\mathsf{RO}}$ as specified in [CJS14].

The security analysis relies on:

1. MuSig2 EUF-CMA security UC-realizing $\mathcal{F}_{\mathsf{IMS}}$ (bound $\epsilon_{\mathsf{IMS}}(\lambda)$)
2. WOTS EUF-OT-CMA security for one-time signatures (bound $\epsilon_{\mathsf{W}}(\lambda)$)
3. IND-CPA security for AEAD encryption with keys from HKDF (bound $\epsilon_{\mathsf{ENC}}(\lambda)$)
4. Zero-knowledge with straight-line extraction in GRO (bound $\epsilon_{\mathsf{SLKE}}(\lambda)$)
5. Trusted party integrity and liveness (bounds ϵ_I, ϵ_L)

$\mathcal{F}_{\mathsf{WISCH}}$ exposes a minimal API with phases init, committed, revealed, completed, two commitments managed by $\mathcal{F}_{\mathsf{B}}$ with timeouts $t_0 < t_1$, selective opening of $S \subseteq [K]$ after t_0, and automatic accept/reject according to an abstract relation R; the functionality reveals only (k, m_k) for opened k and the final decision bits.

Theorem 1 (WISCH UC Security). *Let $\mathcal{F}_{\mathsf{B}}$ be an $(\epsilon_I, \epsilon_L, t)$-secure commitment functionality with timeouts $t_{KO} < t_0 < t_1$. The WISCH protocol π_{WISCH} UC-realizes the ideal functionality $\mathcal{F}_{\mathsf{WISCH}}$ in the $(\mathcal{F}_{\mathsf{IMS}}, \mathcal{F}_{\mathsf{B}})$-hybrid model with global random oracle $\mathcal{G}_{\mathsf{RO}}$. For any PPT adversary $\mathcal{A}$, there exists a PPT simulator $\mathcal{S}$ such that for any PPT environment $\mathcal{Z}$:*

$$\left|\Pr[\mathcal{Z}^{\pi_{\mathsf{WISCH}}, \mathcal{A}, \mathcal{F}_{\mathsf{B}}} = 1] - \Pr[\mathcal{Z}^{\mathcal{F}_{\mathsf{WISCH}}, \mathcal{S}} = 1]\right| \leq \epsilon(\lambda)$$

where $\epsilon(\lambda) = \epsilon_{\mathsf{IMS}}(\lambda) + \epsilon_{\mathsf{SLKE}}(\lambda) + (2^N - 1)K \cdot \epsilon_{\mathsf{ENC}}(\lambda) + \epsilon_I + \epsilon_L$

Definition 4. *Fix any protocol session and its execution up to (but not including) the protocol's reveal step (the point at which the values $\{m(k)\}_{k \in [0, K-1]}$ and the corresponding signing scalars $s(m(k), k)$ become determined). An invocation of HKDF made before this reveal step is called a pre-reveal HKDF query. Let q_H denote the number of pre-reveal HKDF queries made by a party.*

Definition 5. *Let us consider* $y_k := m(k) \parallel k$ *and* $s_k := s(m(k), k) \in \mathbb{Z}_p$ *for* $k \in [0, K-1]$*, and set* $T := \{(s_k, y_k) : k \in [0, K-1]\}$*. We say* EarlyQuery *occurs if the corrupt verifier* $\mathcal{V}^\star$ *makes some pre-reveal query to the HKDF tag that equals an element of* T*.*

Lemma 1 (HKDF freshness bound). *If* q_H *is the number of pre-reveal HKDF queries:* $\Pr[\mathsf{EarlyQuery}] \leq q_H 2^{-\ell_s}$*.*

Proof. Before reveal, the Schnorr scalar $s(m(k), k) \in \mathbb{Z}_p$ is uniformly distributed from $\mathcal{V}^\star$'s view (since honest $\mathcal{P}$'s nonces $r_{P,0}, r_{P,1}$ are uniform and secret). For each of the q_H pre-reveal HKDF queries (s', y'), the probability that $s' = s(m(k), k)$ for some k is at most $K/p < 2^{-\ell_s}$ (where $\ell_s \approx 256$ for secp256k1). By union bound over q_H queries: $\Pr[\mathsf{EarlyQuery}] \leq q_H \cdot 2^{-\ell_s}$.

Theorem 2. *Under verifier-only corruption, for any PPT adversary* $\mathcal{A}$ *controlling* $\mathcal{V}$ *there exists a PPT simulator* $\mathcal{S}_\mathcal{V}$ *such that for any PPT environment* $\mathcal{Z}$*:*

$$\left|\Pr\left[\mathcal{Z}^{\pi_{\mathsf{WISCH}}, \mathcal{A}, \mathcal{F}_\mathsf{B}} = 1\right] - \Pr\left[\mathcal{Z}^{\mathcal{F}_{\mathsf{WISCH}}, \mathcal{S}_V} = 1\right]\right| \leq \epsilon(\lambda).$$

Where $\epsilon(\lambda) = \epsilon_{\mathsf{IMS}}(\lambda) + \epsilon_{\mathsf{ZK}}(\lambda) + q_H 2^{-\ell_s} + \epsilon_I + \epsilon_L$*, and* $\epsilon_{\mathsf{ZK}}(\lambda)$ *is the zero-knowledge simulation error of the NIZK used for encapsulation, and* q_H *is the number of pre-reveal HKDF queries.*

5.3 Correctness

The correctness of WISCH encompasses both completeness and soundness, ensuring that honest parties succeed while cheating attempts are detected with overwhelming probability. These properties are not merely theoretical guarantees but have direct implications for practical deployment. In blockchain applications, for instance, completeness ensures that honest provers can reliably claim collateral locked in smart contracts, while soundness prevents malicious actors from stealing funds through invalid claims.

Completeness requires analyzing the failure modes of honest execution. An honest $\mathcal{P}$ with valid inputs $\{m(k)\}_{k=0}^{K-1}$ satisfying $V(\{m(k)\}) = 1$ can fail only if one of the underlying cryptographic primitives fails: MuSig2 signature generation (bounded by $\epsilon_\mathsf{S}(\lambda)$), Winternitz signature verification (bounded by $\epsilon_\mathsf{W}(\lambda)$), commitment creation in $\mathcal{F}_\mathsf{B}$ (bounded by ϵ_L), or the zero-knowledge proof system (bounded by $\epsilon_{\mathsf{SLKE}}(\lambda)$). The union bound over these failure events yields the completeness error.

Soundness addresses the complementary concern: preventing corrupt provers from succeeding with invalid inputs. The analysis considers two attack vectors. First, a corrupt $\mathcal{P}^\star$ might attempt to forge Schnorr signatures for messages not signed during setup, which is prevented by the EUF-CMA security of MuSig2 and the uniqueness of nonces binding signatures to specific messages. Second, $\mathcal{P}^\star$ might provide valid Schnorr signatures but invalid Winternitz signatures or inputs violating V, which is detected during $\mathcal{V}$'s verification phase.

Theorem 3. *The WISCH protocol achieves:*

1. *Completeness: if $\mathcal{P}$ is honest with inputs $\{m(k)\}_{k=0}^{K-1}$ with $V(\{m(k)\}) = 1$:*

$$\Pr[\textit{Protocol succeeds for } \mathcal{P}] \geq 1 - \epsilon_{\mathsf{S}}(\lambda) - \epsilon_{\mathsf{W}}(\lambda) - \epsilon_L$$

   where we assume the zero-knowledge proof system has perfect completeness, which holds for Σ-protocols transformed via Fischlin [F05].
2. *Soundness: for any PPT adversary $\mathcal{A}$ controlling $\mathcal{P}$ with invalid inputs:*

$$\Pr[\textit{Protocol succeeds for } \mathcal{A}] \leq \epsilon_{\mathsf{S}}(\lambda) + \epsilon_{\mathsf{SLKE}}(\lambda) + \epsilon_{\mathsf{W}}(\lambda) + \epsilon_{\mathsf{nonce}}(\lambda) + \epsilon_I$$

   where $\epsilon_{\mathsf{nonce}}(\lambda) \leq (2^N K)^2/2p$ bounds the probability of nonce collision among the $2^N K$ aggregated nonces.

For corrupt verifiers, we use HKDF programming at reveal time to enable simulation without knowing inputs beforehand. If the corrupt verifier makes q_H pre-reveal HKDF queries, the probability of an early query collision is bounded by $\Pr[\mathsf{EarlyQuery}] \leq q_H 2^{-\ell_s}$. This freshness bound ensures negligible collision probability even under polynomial queries.

6 Conclusions

We presented WISCH, a novel signing protocol achieving efficiency improvements in commitment and verification protocols. By combining Winternitz signatures with jointly-created Schnorr signatures in a nested construction, WISCH reduces verification costs by 90% compared to traditional approaches.

Our main contributions include: a two-layer signature architecture enabling batch verification with selective revelation, a data expansion factor of 35:1 versus 200:1 for standard WOTS, complete UC security proof under standard assumptions, and practical compatibility with existing systems.

The techniques developed—particularly using one signature scheme to encrypt another while maintaining verifiability—open new possibilities in threshold cryptography, privacy-preserving authentication, and distributed systems requiring efficient selective disclosure. WISCH makes complex commitment protocols practical in resource-constrained environments, particularly for blockchain-based multiparty computation.

References

[AHK20] Agrikola, T., Hofheinz, D., Kastner, J: On instantiating the algebraic group model from falsifiable assumptions. In Advances in Cryptology–EUROCRYPT 2020: 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, May 10–14, 2020, Proceedings, Part II 30, pp. 96-126. Springer International Publishing, 2020.

[ADMM14] Andrychowicz, M., Dziembowski, S., Malinowski, D., Mazurek, Ł.: Fair Two-Party Computations via Bitcoin Deposits. Financial Cryptography and Data Security: FC 2014 Workshops, BITCOIN and WAHC 2014, Christ Church, Barbados, March 7, 2014, Revised Selected Papers, pp. 105–121. Springer Berlin Heidelberg.

[AAL+24] Aumayr, L., Avarikioti, Z., Linus, R., Maffei, M., Pelosi, A., Stefo, C., Zamyatin, A.: BitVM: Quasi-Turing Complete Computation on Bitcoin. Cryptology ePrint Archive (2024).

[BOS16] Baum, C., Orsini, E., Scholl, P.: Efficient secure multiparty computation with identifiable abort. Theory of cryptography. Part I, 461–490, Lecture Notes in Comput. Sci., 9985, Springer, Berlin, 2016.

[BDPT25] Baum, C., David, B., Pagnin, E., Takahashi, A.: Universally composable interactive and ordered multi-signatures. Public-key cryptography—PKC 2025. Part II, 3–31, Lecture Notes in Comput. Sci., 15675, Springer, Cham, 2025.

[BDG20] Bellare, M., Davis, H., Günther, F.: Separate your domains: NIST PQC KEMs, oracle cloning and read-only indifferentiability. In Advances in Cryptology–EUROCRYPT 2020: 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, May 10–14, 2020, Proceedings, Part II 30, pp. 3-32. Springer International Publishing, 2020.

[BN06] Bellare, M., Neven, G.: Multi-signatures in the plain public-key model and a general forking lemma. In Proceedings of the 13th ACM conference on Computer and communications security, pp. 390-399. 2006.

[BGW88] Ben-OR, M., Goldwasser, S., Wigderson, A.: Completeness theorems for noncryptographic fault-tolerant distributed computations. In Proceedings of the 20th Annual Symposium on the Theory of Computing (STOC'88), pp. 1-10. 1988.

[BHH+19] Bernstein, D.-J., Hülsing, A., Kölbl, S., Niederhagen, R., Rijneveld, J., Schwabe, P.: The SPHINCS+ signature framework. In Proceedings of the 2019 ACM SIGSAC conference on computer and communications security pp. 2129-2146. 2019.

[BH23] Bindel, N., Hale, B.: A Note on Hybrid Signature Schemes. Cryptology ePrint Archive, Paper 2023/423 https://eprint.iacr.org/2023/423

[BS20] Boneh, D., Shoup, V.: "A graduate course in applied cryptography." Draft 0.5 (2020).

[SEC2] Brown, D.: SEC 2: Recommended elliptic curve domain parameters. https://www.secg.org/sec2-v2.pdf Certicom Res., Mississauga, ON, Canada, Tech. Rep. SEC2-V2 (2010).

[W11] Buchmann, J., Dahmen, E., Ereth, S., Hülsing, A., Rückert, M.: On the security of the Winternitz one-time signature scheme. International Journal of Applied Cryptography 3, no. 1 (2013): 84-96.

[CL04] Camenisch, J., Lysyanskaya, A.: Signature schemes and anonymous credentials from bilinear maps. In Annual international cryptology conference, pp. 56-72. Berlin, Heidelberg: Springer Berlin Heidelberg, 2004.

[CDG+18] Camenisch, J., Drijvers, M., Gagliardoni, T., Lehmann, A., Neven, G.: The wonderful world of global random oracles. Advances in cryptology—EUROCRYPT 2018. Part I, 280–312, Lecture Notes in Comput. Sci., 10820, Springer, Cham, 2018.

[CJS14] Canetti, R., Abhishek, J., Scafuro, A.: Practical UC security with a global random oracle. ACM CCS 2014.

[C20] Canetti, R.: Universally composable security. J. ACM 67 (2020), no. 5, Art. 29, 94 pp.

[CP23] de Castro, L., Peikert, C.: Functional commitments for all functions, with transparent setup and from SIS. Advances in cryptology—EUROCRYPT 2023. Part III, 287–320, Lecture Notes in Comput. Sci., 14006, Springer, Cham, 2023.

[CF13] Catalano, D., Fiore, D.: Vector Commitments and Their Applications. In Public-Key Cryptography – PKC 2013, Lecture Notes in Computer Science, vol 7778, pages 55-72. Springer, Berlin, Heidelberg, 2013.

[CF24] Chiesa, A., Fenzi, G.: zkSNARKs in the ROM with Unconditional UC-Security. In Theory of Cryptography Conference, pp. 67-89. Cham: Springer Nature Switzerland, 2024.

[CKM21] Crites, E., Komlo, C., Maller, M.: How to prove Schnorr assuming Schnorr: Security of multi-and threshold signatures. Cryptology ePrint Archive (2021).

[DN02] Damgård, I., Nielsen, J.-B.: Perfect hiding and perfect binding universally composable commitment schemes with constant expansion factor. In Annual International Cryptology Conference, pp. 581-596. Berlin, Heidelberg: Springer Berlin Heidelberg, 2002.

[F05] Fischlin, M.: Communication-efficient non-interactive proofs of knowledge with online extractors. In Annual International Cryptology Conference, pp. 152-168. Berlin, Heidelberg: Springer Berlin Heidelberg, 2005.

[SYN25] Sedghighadikolaei, K., Yavuz, A., Nouma, S.: Signer-optimal multiple-time post-quantum hash-based signature for heterogeneous IoT Systems. Internet of Things, Volume 33, 2025, 101694, ISSN 2542-6605.

[DPSZ12] Damgård, I., Pastro, V., Smart, N., Zakarias, S.: Multiparty computation from somewhat homomorphic encryption. In Annual cryptology conference, pp. 643-662. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012.

[DRO18] Decker, C., Russell, R., Olaoluwa, O.: eltoo: A simple layer2 protocol for bitcoin. https://blockstream.com/eltoo.pdf (2018).

[DEF+19] Drijvers, M., Edalatnejad, K., Ford, B., Kiltz, E., Loss, J., Neven, G., Stepanovs, I.: On the Security of Two-Round Multi-Signatures. In: 2019 IEEE Symposium on Security and Privacy. doi: 10.1109/SP.2019.00050

[D17] Dryja, T.: Discreet log contracts. https://adiabat.github.io/dlc.pdf (2017).

[FSS+24] Flamini, A., Sciarretta, G., Scuro, M., Sharif, A., Tomasi, A., Ranise, S.: On Cryptographic Mechanisms for the Selective Disclosure of Verifiable Credentials. arXiv:2401.08196 preprint (2024)

[FKL18] Fuchsbauer, G., Klitz, E., Loss, J.: The algebraic group model and its applications. In Advances in Cryptology–CRYPTO 2018: 38th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19–23, 2018, Proceedings, Part II 38, pp. 33-62. Springer International Publishing, 2018.

[FPS20] Fuchsbauer, G., Plouviez, A., Seurin, Y.: Blind Schnorr signatures and signed ElGamal encryption in the algebraic group model. In Advances in Cryptology–EUROCRYPT 2020: 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, May 10–14, 2020, Proceedings, Part II 30, pp. 63-95. Springer International Publishing, 2020.

[GKL24] Garay, J., Kiayias, A., Leonardos, N.: The Bitcoin backbone protocol: analysis and applications. J. ACM 71 (2024), no. 4, Art. 25, 49 pp.

[GW11] Gentry, C., Wichs, D.: Separating succinct non-interactive arguments from all falsifiable assumptions. In Proceedings of the forty-third annual ACM symposium on Theory of computing, pp. 99-108. 2011.

[GKP+22] Ghinea D., Kaczmarczyck, F., Pullman, J., Cretin, J., Kölbl, S., Misoczki, R., Picod, J.-M., Invernizzi, L., Bursztein, E.: Hybrid Post-Quantum Signatures in Hardware Security Keys. Cryptology ePrint Archive, Paper 2022/1225, https://eprint.iacr.org/2022/1225

[GMW87] Goldreich, O., Micali, S., Wigderson, A.: How to play any mental game. In Proceedings of the Nineteenth ACM Symp. on Theory of Computing, STOC, pp. 218-229. New York: ACM, 1987.

[G01] Goldreich, O.: Foundations of cryptography. Volume 1. Cambridge University Press, Cambridge, 2001.

[G04] Goldreich, O.: Foundations of Cryptography, Volume 2. Cambridge: Cambridge university press, 2004.

[GKS23] Grassi, L., Khovratovich, D., Schofnegger, M.: Poseidon2: A faster version of the poseidon hash function. In International Conference on Cryptology in Africa, pp. 177-203. Cham: Springer Nature Switzerland, 2023.

[H13] Hülsing, A.: W-OTS+ - Shorter Signatures for Hash-Based Signature Schemes. In: Youssef, A., Nitaj, A., Hassanien, A.E. (eds) Progress in Cryptology - AFRICACRYPT 2013. AFRICACRYPT 2013. Lecture Notes in Computer Science, vol 7918. Springer, Berlin, Heidelberg.

[KL21] Katz, J., Lindell, Y.: Introduction to modern cryptography. Third edition. Chapman & Hall/CRC Cryptography and Network Security. CRC Press, Boca Raton, FL, 2021.

[L79] Lamport, L.: Constructing digital signatures from a one way function. (1979). Financ Innov 11, 34 (2025). https://doi.org/10.1186/s40854-025-00751-6

[LAM+24] Lerner, S.-D.; Amela, R., Mishra, S., Jonas, M., Álvarez Cid-Fuentes, J.: BitVMX: A CPU for universal computation on Bitcoin. arXiv preprint arXiv:2405.06842 (2024).

[LY10] Libert, B., Yung, M.: Concise Mercurial Vector Commitments and Independent Zero-Knowledge Sets with Short Proofs. In Theory of Cryptography Conference (TCC) 2010, pages 499–517. Springer, 2010.

[MPSW18] Maxwell, G., Poelstra, A., Seurin, Y., Wuille, P.: Simple Schnorr multisignatures with applications to Bitcoin. In: Des. Codes Cryptogr. 87.9 (2019). https://eprint.iacr.org/2018/068.

[M87] Merkle, R.: A digital signature based on a conventional encryption function. In Conference on the theory and application of cryptographic techniques, pp. 369-378. Berlin, Heidelberg: Springer Berlin Heidelberg, 1987.

[NRS21] Nick, J., Ruffing, T., Seurin, Y.: MuSig2: Simple two-round Schnorr multisignatures. In Annual International Cryptology Conference, pp. 189-221. Cham: Springer International Publishing, 2021.

[P91] Pedersen, T.: Non-interactive and information-theoretic secure verifiable secret sharing. In Annual international cryptology conference, pp. 129-140. Berlin, Heidelberg: Springer Berlin Heidelberg, 1991.

[SO25] Serengil, S., Ozpinar, A.: LightDSA: A Python-Based Hybrid Digital Signature Library and Performance Analysis of RSA, DSA, ECDSA and EdDSA in Variable Configurations, Elliptic Curve Forms and Curves. arXiv:2505.23773 preprint (2025) https://arxiv.org/abs/2505.23773

[T14] van Tilborg, H., Jajodia, S.: Encyclopedia of cryptography and security. Springer Science & Business Media, 2014.

[Y82] Yao, A.: Protocols for secure computations. In 23rd annual symposium on foundations of computer science (SFCS 1982), pp. 160-164. IEEE, 1982.

[Y86] Yao, A.: How to generate and exchange secrets. In 27th annual symposium on foundations of computer science (SFCS 1986), pp. 162-167. IEEE, 1986.

[ZZK22] Zhang, C., Zhou, H.-S., Katz, J.: An analysis of the algebraic group model. In International Conference on the Theory and Application of Cryptology and Information Security, pp. 310-322. Cham: Springer Nature Switzerland, 2022.

[AAL24] Aumayr, L., Avarikioti, Z., Linus, R., Maffei, M., Pelosi, A., Stefo, C. and Zamyatin, A., 2024. BitVM: Quasi-Turing Complete Computation on Bitcoin. Cryptology ePrint Archive.

[LAMJC24] Lerner, S.D., Amela, R., Mishra, S., Jonas, M. and Cid-Fuentes, J.Á., 2024. BitVMX: A CPU for universal computation on bitcoin. arXiv preprint arXiv:2405.06842.

[DFP23] Devevey, J., Fallahpour, P., Passelègue, A. and Stehlé, D., 2023, August. A detailed analysis of Fiat-Shamir with aborts. In Annual International Cryptology Conference (pp. 327-357). Cham: Springer Nature Switzerland.

[BDH20] Bellare, M., Davis, H. and Günther, F., 2020, May. Separate your domains: NIST PQC KEMs, oracle cloning and read-only indifferentiability. In Annual International Conference on the Theory and Applications of Cryptographic Techniques (pp. 3-32). Cham: Springer International Publishing.

[FKL18] Fuchsbauer, G., Kiltz, E. and Loss, J., 2018, July. The algebraic group model and its applications. In Annual International Cryptology Conference (pp. 33-62). Cham: Springer International Publishing.

A Formal security definitions

A.1 Security models and analysis

All algorithms and experiments are parameterized by the security parameter 1^λ. To avoid confusion, we leave this dependence implicit in probabilities and experiment names.

Universal composability (UC) framework [C20] and the Global Random Oracle (GRO) model [CJS14] provide a theoretical foundation for security proofs. WISCH's security proof methodology follows the ideal/real paradigm, demonstrating security under the Algebraic One-More Discrete Logarithm (AOMDL) assumption and within the Algebraic Group Model (AGM) [FKL18] as established by the MuSig2 analysis [NRS21]. Winternitz signatures rely on the preimage resistance of hash functions, providing post-quantum security under appropriate parameter choices. This approach ensures composability with other protocols in the ecosystem.

We analyze WISCH under static single-role corruption. The adversary $\mathcal{A}$ must choose at protocol initialization whether to corrupt either the prover or the verifier, but not both. Formally, either $\mathcal{A}$ controls all parties in role $\mathcal{P}$ (with honest verifiers), or $\mathcal{A}$ controls all parties in role $\mathcal{V}$ (with honest provers). This model captures the primary attack vectors in two-party protocols while simplifying the security analysis. Single-role corruption eliminates cross-role attacks where corrupted parties of one type learn from interactions with honest parties of the same type, removing the need for simulation soundness in our zero-knowledge proofs. The corruption choice is static—fixed throughout the protocol execution and across all concurrent sessions.

The global random oracle model The global random oracle model [CJS14] (GRO) addresses composability limitations in the traditional Random Oracle Model by providing a shared, global oracle accessible across protocol boundaries. This formalization is essential for proving security in complex, concurrently executed cryptographic systems. This model is given by an ideal functionality $\mathcal{G}_{\mathsf{RO}}$ providing:

1. Global accessibility: All parties (including adversaries and simulators) access the same instance of $\mathcal{G}_{\mathsf{RO}}$.
2. Consistency: Identical queries return identical outputs, regardless of protocol context.

We use GRO as described in [BDPT25, Section 2]. The functionality modelling it follows:

Algorithm 4 GRO.QUERY(m)

Parameters: Output size function $\ell(\lambda)$
State: Initially empty list List_H
procedure QUERY
  Upon receiving $(\mathsf{HashQuery}, m)$ from party (P, sid):
  Search for $(m, h) \in \mathsf{List}_H$
  if no such h exists **then**
    Sample $h \xleftarrow{\$} \{0,1\}^{\ell(\lambda)}$
    Set $\mathsf{List}_H = \mathsf{List}_H \cup (m, h)$
  end if
  Output $(\mathsf{HashConfirm}, h)$ to (P, sid)
end procedure

A.2 Security concepts and assumptions

Basic concepts

Definition 6 (Algebraic adversary). *An adversary $\mathcal{A}$ is algebraic if whenever it outputs a group element $Y \in G$, it also provides an algebraic representation $(\alpha, \{\beta_i\}_{i=1}^n)$ such that: $Y = g^\alpha \prod_{i=1}^n X_i^{\beta_i}$, where $X_1, \ldots, X_n$ are all group elements that $\mathcal{A}$ has received as input or computed through oracle queries up to that point. Here $\alpha \in \mathbb{Z}_p$ and $\beta_i \in \mathbb{Z}_p$ for all $i \in \{1, \ldots, n\}$.*

This research will assume all adversaries $\mathcal{A}$ to be algebraic, what implies that we will be working under the algebraic group model (AGM) as described by Fuchsbauer, Kiltz and Loss in [FKL18].

Definition 7 (Knowledge extractor). *A knowledge extractor $\mathfrak{E}$ for relation R is a PPT algorithm such that, for any PPT prover $\mathcal{P}^\star$ and instance x, whenever $\mathcal{P}^\star$ convinces the verifier on x, $\mathfrak{E}^{\mathcal{P}^\star}(x)$ outputs w with $(x, w) \in R$ except with negligible probability.*

Definition 8 ((ϵ, t)-EUF-OT-CMA security). *A digital signature scheme Σ is (ϵ, t)-existentially unforgeable under one-time chosen message attacks if for any adversary $\mathcal{A}$ running in time at most t and making at most one signing query:*

$$\Pr\left[\mathsf{Exp}^{\mathsf{EUF-OT-CMA}}_{\Sigma,\mathcal{A}}(\lambda) = 1\right] \leq \epsilon.$$

See [KL21, Chapter 13] for more details about the game $\mathsf{Exp}^{\mathsf{EUF-OT-CMA}}_{\Sigma,\mathcal{A}}$.

Definition 9 ((ϵ, t)-EUF-CMA security). *A multi-signature scheme given by the tuple $\mathsf{MS} = (\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{KeyAgg}, \mathsf{Sign}, \mathsf{SignAgg}, \mathsf{Verify})$ with n signers is (ϵ, t)-EUF-CMA secure if for any adversary $\mathcal{A}$ controlling at most $n-1$ parties and running in time at most t:*

$$\Pr\left[\mathsf{Exp}^{\mathsf{MS-EUF-CMA}}_{\mathsf{MS},\mathcal{A},n}(\lambda) = 1\right] \leq \epsilon$$

where the experiment $\mathsf{Exp}^{\mathsf{MS-EUF-CMA}}_{\mathsf{MS},\mathcal{A},n}(\lambda)$ is defined as:

1. $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$
2. *$\mathcal{A}$ specifies corrupted parties $\mathcal{C} \subset \{1, \ldots, n\}$ with $|\mathcal{C}| < n$*
3. *For each honest party $i \notin \mathcal{C}$: $(\mathsf{sk}_i, \mathsf{pk}_i) \leftarrow \mathsf{KeyGen}(\mathsf{pp})$*
4. *$\mathcal{A}$ provides public keys $\{\mathsf{pk}_j\}_{j \in \mathcal{C}}$ for corrupted parties*
5. *$(m^\star, \sigma^\star) \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{sign}}}(\mathsf{pp}, \{\mathsf{pk}_i\}_{i=1}^n)$*
6. *Return 1 if:*
   - *$\mathsf{Verify}(\mathsf{pp}, \{\mathsf{pk}_i\}_{i=1}^n, m^\star, \sigma^\star) = 1$*
   - *$m^\star$ was not queried to $\mathcal{O}_{\mathsf{sign}}$ for the full signer set $\{1, \ldots, n\}$*

where $\mathcal{O}_{\mathsf{sign}}$ allows $\mathcal{A}$ to initiate concurrent signing sessions, providing responses for honest parties according to the protocol.

Definition 10 ((ϵ, t)-IND-CPA security). *An encryption scheme given by the tuple $\mathcal{E} = (\mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec})$ is (ϵ, t)-IND-CPA secure if for any adversary $\mathcal{A}$ running in time at most t:*

$$\left|\Pr\left[\mathsf{Exp}^{\mathsf{IND-CPA}-1}_{\mathcal{E},\mathcal{A}}(\lambda) = 1\right] - \Pr\left[\mathsf{Exp}^{\mathsf{IND-CPA}-0}_{\mathcal{E},\mathcal{A}}(\lambda) = 1\right]\right| \leq \epsilon$$

where $\mathsf{Exp}^{\mathsf{IND-CPA}-b}_{\mathcal{E},\mathcal{A}}(\lambda)$ encrypts m_b in the challenge phase.

Definition 11 ((ϵ, t)-discrete logarithm). *Let GrGen be a group generation algorithm that outputs (G, q, g) where G is a cyclic group of prime order q with generator g. The discrete logarithm problem is (ϵ, t)-hard relative to GrGen if for any adversary $\mathcal{A}$ running in time at most t:*

$$\Pr\left[\begin{array}{c}(G, q, g) \leftarrow \mathsf{GrGen}(1^\lambda),\ x \xleftarrow{\$} \mathbb{Z}_p,\ X := g^x \\ x' \leftarrow \mathcal{A}(G, q, g, X)\end{array} : x' = x\right] \leq \epsilon$$

Definition 12 ((ϵ, t)-one-more discrete logarithm). *The one-more discrete logarithm problem is (ϵ, t)-hard relative to GrGen if for any adversary $\mathcal{A}$ running in time at most t and making at most n discrete logarithm oracle queries:*

$$\Pr\left[\begin{array}{cc}\mathbb{G} = (G, q, g) \leftarrow \mathsf{GrGen}(1^\lambda) & \\ X_1, \ldots, X_{n+1} \xleftarrow{\$} G & : \forall i \in [n+1] : g^{x_i} = X_i \\ (x_1, \ldots, x_{n+1}) \leftarrow \mathcal{A}^{\mathsf{DLog}_g(\cdot)}(\mathbb{G}, X_1, \ldots, X_{n+1}) & \end{array}\right] \leq \epsilon$$

Definition 13 ((ϵ, t)-preimage resistance). *A hash function $H : \mathcal{D} \to \mathcal{R}$ is (ϵ, t)-preimage resistant if for any adversary $\mathcal{A}$ running in time at most t:*

$$\Pr\left[y \xleftarrow{\$} \mathcal{R},\ x \leftarrow \mathcal{A}(y) : H(x) = y\right] \leq \epsilon$$

Definition 14. *An environment $\mathcal{Z}$ is a probabilistic polynomial-time algorithm that: Controls the inputs to all protocol parties, observes all outputs from protocol parties, coordinates with the adversary $\mathcal{A}$, attempts to distinguish between real and simulated protocol executions, and outputs a single bit indicating its distinguishing decision.*

Zero-knowledge with straight-line extractability Since our protocol operates under concurrent composition in the UC framework, we require zero-knowledge proof systems Π with straight-line knowledge extraction to avoid rewinding issues.

Remark 5. The straight-line extraction property above is satisfied, for example, by applying the Fischlin transformation [F05] to a Σ-protocol with unique accepting responses, instantiated in the GRO model with domain-separated queries. The resulting extractor is straight-line: it interacts without rewinding, observes the prover's oracle queries, and may program fresh inputs to the global oracle as needed. In this setting, the construction is compatible with concurrent composition in the UC framework with a global setup.

A.3 Environments

Environments model all possible contexts in which the protocol might be deployed, including other protocols running concurrently and external system interactions. In the WISCH situation we have the following protocol-specific environments:

Definition 15. *In the real world execution* $\mathcal{Z}^{\pi_{\mathsf{WISCH}},\mathcal{A},\mathcal{F}_B}$*:*

1. *Environment* $\mathcal{Z}$ *provides inputs to honest parties running protocol* π_{WISCH}*.*
2. *Adversary* $\mathcal{A}$ *controls corrupted parties and executes the real protocol.*
3. *Both honest parties and* $\mathcal{A}$ *have access to the trusted verification service functionality* $\mathcal{F}_B$*.*
4. $\mathcal{Z}$ *observes:*
   - *All messages sent by honest parties.*
   - *All outputs produced by honest parties.*
   - *All communication between* $\mathcal{A}$ *and corrupted parties.*
   - *All interactions with* $\mathcal{F}_B$*: commitment creation, witness submission, resolution queries.*
5. $\mathcal{Z}$ *can adaptively provide new inputs based on observed protocol execution.*

Definition 16. *In the ideal world execution* $\mathcal{Z}^{\mathcal{F}_{\mathsf{WISCH}},\mathcal{S}}$*:*

1. *Environment* $\mathcal{Z}$ *provides the same inputs to ideal functionality* $\mathcal{F}_{\mathsf{WISCH}}$*.*
2. *Simulator* $\mathcal{S}$ *replaces adversary* $\mathcal{A}$ *and interacts with* $\mathcal{F}_{\mathsf{WISCH}}$*.*
3. $\mathcal{Z}$ *observes:*
   - *Outputs from* $\mathcal{F}_{\mathsf{WISCH}}$ *(commitment resolutions, resolution of challenges).*
   - *Simulated protocol messages generated by* $\mathcal{S}$*.*
   - *Simulated interactions created by* $\mathcal{S}$ *(without access to* $\mathcal{F}_{\mathsf{B}}$*).*
4. $\mathcal{S}$ *generates a view indistinguishable from the real world without access to:*
   - *Honest party secrets.*
   - *The functionality* $\mathcal{F}_{\mathsf{B}}$*.*
   - *The actual inputs of honest parties (until revealed by* $\mathcal{F}_{\mathsf{WISCH}}$*).*

Definition 17. *For an environment* $\mathcal{Z}$*, the distinguishing advantage is defined by:*

$$\mathbf{Adv}^{sim}_{\mathcal{Z}}(\lambda) = \left|\Pr[\mathcal{Z}^{\pi_{\mathsf{WISCH}},\mathcal{A},\mathcal{F}_{\mathsf{B}}} = 1] - \Pr[\mathcal{Z}^{\mathcal{F}_{\mathsf{WISCH}},\mathcal{S}} = 1]\right|$$

where:

1. $\Pr[\mathcal{Z}^{\pi_{\mathsf{WISCH}},\mathcal{A},\mathcal{F}_{\mathsf{B}}} = 1]$ *is the probability* $\mathcal{Z}$ *outputs 1 in the real world with access to* $\mathcal{F}_{\mathsf{B}}$*.*
2. $\Pr[\mathcal{Z}^{\mathcal{F}_{\mathsf{WISCH}},\mathcal{S}} = 1]$ *is the probability* $\mathcal{Z}$ *outputs 1 in the ideal world.*

A.4 Simulators

Simulator for corrupt prover The goal of this simulator is to simulate honest $\mathcal{V}$'s view for corrupt $\mathcal{P}^\star$. It is described in Algorithm 5.

Remark 6. The simulator never enumerates $\{0,1\}^N$: it uses lazy sampling via EnsureEntry and straight-line extraction (Alg. 12). Therefore, its running time is $\mathrm{poly}(\lambda, K, q_{\mathsf{RO}}, q_{\mathsf{IMS}})$.

Algorithm 5 Simulator $\mathcal{S}_{\mathcal{P}}$ for corrupt prover

procedure SIMULATEMUSIG2
  Maintain empty maps R, s keyed by (m, k)
  Oracle ENSUREENTRY(m, k):
  if $(m, k) \notin \mathrm{dom}(\mathsf{R})$ **then**
    Act as honest in $\mathcal{F}_{\mathsf{IMS}}$ for message m and index k to obtain $(R(m, k), s(m, k))$
    Store $(R(m, k), s(m, k))$ into R, s
  end if
  return $(\mathsf{R}[m, k], \mathsf{s}[m, k])$
end procedure
procedure SIMULATEPROTOCOL
  Receive hints $\{hs(m, k), H(m), s'(m)\}$ from $\mathcal{P}^\star$
  Verify hints are well-formed
  Defer batch checks to openings; record $(hs(m, \cdot), H(m), s'(m))$ and, when needed, call ENSUREENTRY(m, k) for those (m, k) only
  Receive encrypted packages $\{E(m, k)\}_{m,k}$ from $\mathcal{P}^\star$
end procedure
procedure EXTRACTINPUTS
  if $\mathcal{P}^\star$ provides ZKP **then**
    Run knowledge extractor to obtain:
      Inputs $\{m(k)\}_{k=0}^{K-1}$
      All Schnorr signatures $\{s(m, k)\}_{m,k}$
      All Winternitz signatures $\{W(m, k)\}_{m,k}$
    Verify consistency:
      $\forall m, k : k_{\mathsf{enc}}(m, k) = \mathsf{HKDF}(s(m, k), m \parallel k)$
      $\forall m, k : E(m, k) := \mathsf{Enc}^{\mathsf{ad}(sid,k,\mathsf{pub}(k))}_{k_{\mathsf{enc}}(m,k)}\big(W(m, k)\big)$
    Forward extracted inputs to $\mathcal{F}_{\mathsf{WISCH}}$
  end if
end procedure
procedure SIMULATEBOB
  Simulate `commit` creation
  Monitor $\mathcal{P}^\star$'s commitments submitted to $\mathcal{F}_{\mathsf{B}}$
  if valid K signatures received **then** Forward to $\mathcal{F}_{\mathsf{WISCH}}$
  Simulate challenges if needed
end procedure

Simulator for corrupt verifier The goal of this simulator is simulate honest $\mathcal{P}$'s view for corrupt $\mathcal{V}^\star$. It is described in Algorithm 6.

Algorithm 6 Simulator $\mathcal{S}_\mathcal{V}$ for corrupt verifier

procedure SETUP
  Generate real Winternitz public keys $\{\mathsf{pub}_i(k)\}_{i,k}$
  Act as honest party in $\mathcal{F}_{\mathsf{IMS}}$ for MuSig2 protocol
end procedure
procedure SIMULATECOMMITMENT
  for each $(m,k) \in \{0,1\}^N \times [0, K-1]$ **do**
    $k_{\mathsf{pre}}(m,k) \xleftarrow{\$} \{0,1\}^{\kappa_{\mathsf{KDF}}}$
    $W(m,k) \leftarrow \text{WOTS.SIGN}(\mathsf{priv}(k), m)$
    $C(m,k) \leftarrow \mathsf{Enc}^{\mathsf{ad}(sid,k,\mathsf{pub}(k))}_{k_{\mathsf{pre}}(m,k)}\big(W(m,k)\big)$
    send $C(m,k)$ to $\mathcal{V}^\star$
  end for
  simulate the ZK proof shown to $\mathcal{V}^\star$
end procedure
procedure SIMULATEZKP
  Use ZKP simulator to prove knowledge of:
    Consistent signatures and encryptions
    Correct HKDF derivations: $k_{\mathsf{enc}}(m,k) = \mathsf{HKDF}(s(m,k), m \parallel k)$
end procedure
procedure SIMULATEREVELATION$(\{m(k)\}_{k=0}^{K-1})$
  for $k = 0$ **to** $K-1$ **do**
    submit $(R(m(k),k), s(m(k),k))$ to $\mathcal{F}_{\mathsf{B}}$
    program $\mathsf{HKDF}(s(m(k),k),\, m(k) \,\|\, k) := k_{\mathsf{pre}}(m(k),k)$
    $W(m(k),k) \leftarrow \mathsf{Dec}^{\mathsf{ad}(sid,k,\mathsf{pub}(k))}_{k_{\mathsf{enc}}(m(k),k)}\big(C(m(k),k)\big)$
    Verify locally as in the real protocol
  end for
end procedure

Remark 7. The loop over (m,k) mirrors the real protocol's offline emission of all $\{C(m,k)\}$. Under our parameter regime (N fixed with $2^N \leq \text{poly}(\lambda)$, concretely $N \in \{4,8\}$), this is polynomial in (λ, K).

On the functionality $\mathcal{F}_{\mathsf{IMS}}$ MuSig2 fulfills [BDPT25, definition 5 and definition 6]. Furthermore, MuSig2 is EUF-CMA secure in the ROM model [NRS21], what implies that it is IMS-UF-CMA secure using Baum's notation. [BDPT25, Theorem 7] implies that MuSig2 UC-realizes $\mathcal{F}_{\mathsf{IMS}}$ in $\mathcal{G}_{RO}$.

B Complete protocol specification

Realization sketch of $\mathcal{F}_{\mathsf{WISCH}}$

Algorithm 7 Protocol π_{WISCH}: Detailed internal logic

State: *state* $\in \{\texttt{init}, \texttt{setup}, \texttt{committed}, \texttt{revealed}, \texttt{completed}\}$
  inputs $\{m(k)\}_{k=0}^{K-1}$, session with $\mathcal{F}_{\mathsf{IMS}}$
procedure INITIALIZE($\mathcal{P}$, $\mathcal{V}$, parameters)
  if matching parameters received from both parties **then**
    Initialize MuSig2 session with $\mathcal{F}_{\mathsf{IMS}}$ for $2^N \times K$ signatures
    state $\leftarrow$ `setup`
  end if
end procedure
procedure SETUP
  Generate Winternitz public keys $\{\mathsf{pub}_i(k)\}_{i,k}$ and submit
  for $(m, k) \in \{0,1\}^N \times [0, K-1]$: Execute MuSig2 setup via $\mathcal{F}_{\mathsf{IMS}}$
  Handle hint transmission and batch verification
  if Successful ZKP received from $\mathcal{P}$ with inputs $\{m(k)\}_{k=0}^{K-1}$ **then**
    Store commitment
    Create $\texttt{commit}_0$ (K-checksig, timeout t_0) via $\mathcal{F}_{\mathsf{B}}$
    Create $\texttt{commit}_1$ (Winternitz + V, timeout t_1) via $\mathcal{F}_{\mathsf{B}}$
    state $\leftarrow$ `committed`
  end if
end procedure
procedure COMMITMENT
  if *CurrentTime* $\geq t_{KO}$ **then**
    Query $\mathcal{F}_{\mathsf{IMS}}$ for signatures on each $m(k)$
    Submit K signatures to commitment $\texttt{commit}_0$ handled by $\mathcal{F}_{\mathsf{B}}$
    if timeout t_0 reached: $\mathcal{V}$ wins
  end if
end procedure
procedure REVELATION Extract inputs $\{m(k)\}_{k=0}^{K-1}$ from $\texttt{commit}_0$ via nonce correspondence
  For each $k \in \{0, \ldots, K-1\}$**:**
    Retrieve Schnorr signature $s(m(k), k)$
    Derive decryption key: $k_{\mathsf{enc}}(m(k), k) = \mathrm{HKDF}(s(m(k), k), m(k) \parallel k)$
    Decrypt Winternitz signature: $W(m(k), k) = D_{k_{\mathsf{enc}}(m(k),k)}(E(m(k), k))$
  Send revealed inputs and decrypted signatures to honest $\mathcal{V}$
  Verify Winternitz signatures and program predicate $V(\{m(k)\})$
  state $\leftarrow$ `revealed`
end procedure
procedure CHALLENGE($\mathcal{V}$, *type*, *witness*)
  if received before t_1 **then**
    Submit challenge to $\mathcal{F}_B$ who checks via $\texttt{commit}_1$
    if challenge is valid: $\mathcal{V}$ wins
  end if
end procedure
procedure COMPLETE
  if timeout t_1 reached **then** $\mathcal{P}$ wins **and** *state* $\leftarrow$ `completed`
end procedure

The trusted third party functionality $\mathcal{F}_{\mathsf{B}}$ The functionality $\mathcal{F}_{\mathsf{B}}$ (Algorithm 8) captures the role of a trusted party (Bob) in charge of verifying the commitments and results of the challenges. In applications, it could be representing the role of a blockchain. It captures the features that will be required by the forthcoming WISCH functionality (Algorithm 3).

Algorithm 8 Functionality $\mathcal{F}_{\mathsf{B}}$ (Trusted Commitment Service)

Session: Each instance is parameterized by *sid*
Parties: Prover $\mathcal{P}$, Verifier $\mathcal{V}$, Adversary $\mathcal{A}$
State: *Commits*: map $cid \mapsto (parties, timeout, winner, pred, resolved, witnesses)$
  Clock: monotonic time, initially 0 (advanced by $\mathcal{A}$, subject to $\Delta t \leq \Delta$ per tick)
procedure CreateCommitment(*sid*, *parties*, *timeout*, *winner*, *pred*)
  Upon (Create, *sid*, *parties*, *timeout*, *winner*, *pred*) from $P \in parties$:
  Generate fresh *cid*; store $Commits[cid] \leftarrow (parties, timeout, winner, pred, \perp, \perp)$
  Leak (Created, *sid*, *cid*, *timeout*) to $\mathcal{A}$
  return *cid* to all parties in *parties*
end procedure
procedure SubmitWitness(*sid*, *cid*, *party*, *witness*)
  Upon (Submit, *sid*, *cid*, *witness*) from *party*:
  Require $party \in Commits[cid].parties$ and $resolved = \perp$
  Store $Commits[cid].witnesses \leftarrow witness$
  Leak (Submitted, *sid*, *cid*, $|witness|$) to $\mathcal{A}$ ▷ Size only
  Evaluate $Commits[cid].pred(witness)$; if true, set $resolved \leftarrow party$
end procedure
procedure AdvanceTime(Δt)
  Upon (Tick, Δt) from $\mathcal{A}$ with $\Delta t \leq \Delta$:
  $Clock \leftarrow Clock + \Delta t$
  for each *cid* with $Clock \geq timeout$ and $resolved = \perp$ **do**
    Set $resolved \leftarrow winner$; notify all parties
  end for
end procedure
procedure Query(*sid*, *cid*)
  Upon (Query, *sid*, *cid*) from any party:
  return $(Clock, Commits[cid].resolved, Commits[cid].witnesses)$
end procedure

Remark 8 (Instantiation and error bounds). $\mathcal{F}_{\mathsf{B}}$ is an idealized functionality with deterministic correctness. The predicate *pred* is specified at commitment creation and evaluated on submitted witnesses (e.g., Schnorr signature verification for $\texttt{commit}_0$, or WOTS and program verification for $\texttt{commit}_1$ in the WISCH protocol). When instantiated by a real system such as a blockchain with bounded reorganization, the integrity and liveness bounds ϵ_I, ϵ_L capture the gap between ideal and real behavior: ϵ_I bounds incorrect predicate evaluation, and ϵ_L bounds failure to resolve within the timeout. Under standard assumptions (honest majority, bounded delay Δ), both are negligible [GKL24].

Definition 18. *The functionality $\mathcal{F}_{\mathsf{B}}$ satisfies (ϵ, t)-integrity if for any PPT adversary $\mathcal{A}$ running in time at most t:*

$$\Pr\left[\begin{array}{c}\mathcal{F}_{\mathsf{B}} \text{ resolves commitment cid in favour of } \mathcal{P} \\ \text{while conditions for } \mathcal{P} \text{ are not satisfied}\end{array}\right] \leq \epsilon$$

In particular, commitments can only be created with consent of all specified parties; resolution outcomes follow the verification results or timeout rules deterministically; the verification of witnesses follows the commitment's verification logic, and once resolved, a commitment cannot change its resolution.

Definition 19. *The functionality $\mathcal{F}_{\mathsf{B}}$ satisfies (ϵ, t)-liveness if for all honest parties following the protocol, within time bound t:*

$$\Pr[\textit{Every commitment reaches a definite resolution}] \geq 1 - \epsilon$$

where a definite resolution means either that the commitment resolves based on satisfied conditions before timeout, or that the timeout is reached and the commitment resolves according to timeout rules.

Definition 20. *The functionality $\mathcal{F}_{\mathsf{B}}$ satisfies finality if:*

1. *Once $\mathit{CommitSet}[\mathit{cid}].\mathit{resolved} \neq \perp$, it never changes.*
2. *If $\mathit{CurrentTime} = t$ then $\mathit{CurrentTime} \geq t$ after any operation sequence.*
3. *All parties querying $\mathcal{F}_{\mathsf{B}}$ observe the same commitment resolution.*

Definition 21. *The functionality $\mathcal{F}_{\mathsf{B}}$ is an $(\epsilon_I, \epsilon_L, t)$-secure commitment functionality if it satisfies (ϵ_I, t)-integrity, (ϵ_L, t)-liveness, and finality.*

Remark 9. The functionality $\mathcal{F}_{\mathsf{B}}$ can be specialized into a blockchain functionality $\mathcal{F}_{\mathsf{B}}$ due to the existence of a bijection between the following concepts, which preserve the essential properties:

1. Commitments become UTXOs.
2. Commitment operations correspond to transactions.
3. Abstract verification corresponds to script execution.

B.1 Protocol description

The WISCH protocol enables a prover $\mathcal{P}$ to commit to inputs $\{m(k)\}_{k=0,\ldots,K-1}$ that satisfy a predicate V agreed upon with $\mathcal{V}$. The protocol's key innovation is using jointly-created Schnorr signatures to encrypt Winternitz signatures, providing both efficiency and security.

Protocol overview At a high level, WISCH operates in three phases:

1. **Setup Phase**: $\mathcal{P}$ generates K Winternitz key pairs and publishes the public keys. $\mathcal{P}$ and $\mathcal{V}$ jointly create $2^N \times K$ Schnorr signatures using MuSig2, where $\mathcal{P}$ encrypts his Winternitz signatures with the resulting Schnorr signatures. A zero-knowledge proof ensures $\mathcal{P}$ commits to specific inputs.
2. **Commitment Phase**: After time t_{KO}, $\mathcal{P}$ reveals his chosen inputs by submitting the corresponding Schnorr signatures to $\mathcal{F}_{\mathsf{B}}$. This implicitly reveals $\{m(k)\}_{k=0,\dots,K-1}$ due to the unique binding between nonces and messages.
3. **Verification Phase**: $\mathcal{V}$ decrypts the Winternitz signatures using the revealed Schnorr signatures and verifies both their validity and $V(\{m(k)\}) = 1$. She can challenge before timeout t_1 if any verification fails.

Batch WOTS signature creation The prover $\mathcal{P}$ creates K private Winternitz keys (one for each step of the challenge), he computes their public keys and submits them. For each $k \in \{0, \dots, K-1\}$, let $\mathsf{priv}(k)$ be the $WOTS$ private key generated by $\mathcal{P}$ for step k, which is a string composed of ℓ (n-bit) numbers $\mathsf{priv}_i(k)$: $\mathsf{priv}(k) = (\mathsf{priv}_0(k), \dots, \mathsf{priv}_{\ell-1}(k))$, and likewise with $\mathsf{pub}(k)$, i.e. $\mathsf{pub}_i(k) = {\mathrm{H}_{\mathsf{win}}}^{15}(\mathsf{priv}_i(k))$, for $i = 0, \dots, \ell - 1$.

Definition 22 (Winternitz array). *Let $W(m,k)$ be the WOTS signature of the message m in step k, i.e. $W(m,k)$ is obtained by signing the number $m \in \{0,1\}^N$ with the private key $\mathsf{priv}(k)$. Explicitly,*

$$W(m,k) = ({\mathrm{H}_{\mathsf{win}}}^{b_0(m)}(\mathsf{priv}_0(k)), \dots, {\mathrm{H}_{\mathsf{win}}}^{b_{\ell-1}(m)}(\mathsf{priv}_{\ell-1}(k))) \in \{0,1\}^{\kappa\ell}$$

where $b(m) = m \parallel c$ is obtained concatenating the hexadecimal expression of m with the hexadecimal expression of its checksum. We can think of this large set of $2^N \times K$ signatures as an array whose entries lie in $\{0,1\}^{n\ell}$.

$$\begin{matrix} W(0,0) & W(0,1) & \cdots & W(0,K-1) \\ W(1,0) & W(1,1) & \cdots & W(1,K-1) \\ \vdots & \vdots & \vdots & \vdots \\ W(15,0) & W(15,1) & \cdots & W(15,K-1) \end{matrix}$$

where the k-column represents the signatures of all numbers in $\{0,1\}^N$ with the k-th Winternitz private key, only one of them to be used for each commitment operation step. Note also that for each k, the place $W(m(k),k)$ is the WOTS signature of input $m(k)$ at step k, thus following and adequate path from left to right, we get all the signatures of the $m(k)$.

Joint Schnorr signatures In this setup stage, $\mathcal{P}$ and $\mathcal{V}$ already convened on:

1. A joint signing protocol: Schnorr with group generator $g \in E$ over $\mathbb{Z}_p$, with signature combination by means of the protocol MuSig2.

2. Hash functions $H_* : \{0,1\}^\star \to \mathbb{Z}_p$ for creation of Schnorr signatures, combining the public keys of the signatures, combining the signatures in MuSig2 and generating a linear combination of the signatures.
3. We use an authenticated encryption scheme with associated data:

$$E(m,k) \;:=\; \mathsf{Enc}^{\mathsf{ad}(\mathrm{sid},k,\mathsf{pub}(k))}_{k_{\mathsf{enc}}(m,k)}\big(W(m,k)\big),$$

where $k_{\mathsf{enc}}(m,k) \;=\; \mathsf{HKDF}\big(s(m,k),\, m\|k\big)$. Decryption uses Dec with the same associated data:

$$W = \mathsf{Dec}^{\mathsf{ad}(\mathrm{sid},k,\mathsf{pub}(k))}_{k_{\mathsf{enc}}(m(k),k)}\big(E(m(k),k)\big),$$

returning $\perp$ on failure. Let the corresponding advantage be $\epsilon_{\mathsf{ENC}}(\lambda)$. We bind each ciphertext to its public context using associated data

$$\mathsf{ad}(sid,k,\mathsf{pub}(k)) := \langle sid\rangle \,\|\, \langle k\rangle \,\|\, \langle \mathsf{pub}(k)\rangle,$$

where $\langle\cdot\rangle$ denotes encoding. The associated data is public and identical at encryption and decryption; it serves to bind $E(m,k)$ to its session and slot, and to the corresponding public key, without revealing m. Domain separation for other primitives is as already modeled in the paper: uses of $\mathrm{H}_{\mathsf{sig}}$, $\mathrm{H}_{\mathsf{non}}$, and HKDF are treated as disjoint domains of the global random oracle.
4. A key derivation function $\mathsf{HKDF} : \{0,1\}^\star \to \{0,1\}^{\ell_{\mathrm{key}}}$, where ℓ_{key} is the encryption key length, modeled as a random oracle in the GRO model $\mathcal{G}_{\mathsf{RO}}$.

Remark 10. Let us note that, since we work in the GRO model, we treat $\mathrm{H}_{\mathsf{sig}}$, $\mathrm{H}_{\mathsf{non}}$, $\mathrm{H}_{\mathsf{agg}}$, and HKDF as independent global random oracles. In particular, they could be instantiated with a single global oracle H with domain-separation tags.

We realize HKDF via the GRO with its own domain-separation tag. In our proofs we will *program* HKDF at reveal-time, but only on inputs that are *fresh* (i.e., no party has queried the HKDF tag at those inputs before reveal).

Setup ceremony Let $x_P, x_V \in \mathbb{Z}_p^\star$ be the respective Schnorr secret keys of $\mathcal{P}$ and $\mathcal{V}$, and let X be the aggregated joint public signature. We use the same index convention for all other nonces involved in the protocol. The prover $\mathcal{P}$ and the verifier $\mathcal{V}$ have two round of nonce exchanges, but on each round instead of only two nonces they exchange $2 \times 2^N \times K$ nonces *(with fixed N; for instance, $N = 8$ so $2^N = 256$).*

The protocol π_{WISCH} is described in Algorithms 9 and 10. The commitments $\mathtt{commit}_i$ in this protocol are handled by the trusted party Bob, formalized in Functionality $\mathcal{F}_B$ (Algorithm 8).

Remark 11 (Batch verification usage). The batch verification equation

$$g^{s'(m)} \stackrel{?}{=} \prod_{k=0}^{K-1} X^{c(m,k)l(m,k)} \prod_{k=0}^{K-1} R(m,k)^{l(m,k)}$$

is used in two distinct contexts:

Algorithm 9 Protocol π_{WISCH}

Require: Parameters K, N, κ, w, verification predicate V, timeouts $t_{KO} < t_0 < t_1$
Require: Parties: $\mathcal{P}$, $\mathcal{V}$, Verification service $\mathcal{F}_{\mathsf{B}}$

Setup Phase:
Step 1: Winternitz Key Generation
for $k \leftarrow 0$ to $K-1$ **do**
  for $i \leftarrow 0$ to $\ell - 1$ **do** $\mathsf{priv}_i(k) \xleftarrow{\$} \{0,1\}^\kappa$; $\mathsf{pub}_i(k) = \mathrm{H}_{\mathsf{win}}{}^{2^w-1}(\mathsf{priv}_i(k))$
  end for
  $\mathsf{priv}(k) = (\mathsf{priv}_0(k), \ldots, \mathsf{priv}_{\ell-1}(k))$; $\mathsf{pub}(k) = (\mathsf{pub}_0(k), \ldots, \mathsf{pub}_{\ell-1}(k))$
end for
$\mathcal{P}$ submits all $\mathsf{pub}(k)$
Step 2: MuSig2 Setup
$\mathcal{P}$: $x_P \xleftarrow{\$} \{1, 2, \ldots, p-1\}$, compute $X_P = g^{x_P}$
$\mathcal{V}$: $x_V \xleftarrow{\$} \{1, 2, \ldots, p-1\}$, compute $X_V = g^{x_V}$
Both: $L = \mathrm{H}_{\mathsf{agg}}(X_P \parallel X_V)$; $a_P = \mathrm{H}_{\mathsf{agg}}(L \parallel P \parallel X_P)$, and $a_V = \mathrm{H}_{\mathsf{agg}}(L \parallel V \parallel X_V)$
Aggregated key: $X = X_P^{a_P} X_V^{a_V}$
Step 3: Batch Nonce Generation
for each $m \in \{0,1\}^N$ **do**
  for $k \leftarrow 0$ to $K-1$ **do**
    $\mathcal{P}$: $r_{P,0}(m,k), r_{P,1}(m,k) \xleftarrow{\$} \{1, 2, \ldots, p-1\}$
    $\mathcal{P}$ computes and sends: $R_{P,0}(m,k) = g^{r_{P,0}(m,k)}$, $R_{P,1}(m,k) = g^{r_{P,1}(m,k)}$
    $\mathcal{V}$: $r_{V,0}(m,k), r_{V,1}(m,k) \xleftarrow{\$} \{1, 2, \ldots, p-1\}$
    $\mathcal{V}$ computes and sends: $R_{V,0}(m,k) = g^{r_{V,0}(m,k)}$, $R_{V,1}(m,k) = g^{r_{V,1}(m,k)}$
    Both compute: $R_j(m,k) = R_{P,j}(m,k) R_{V,j}(m,k)$ for $j \in \{0,1\}$
    Both: $n(m,k) = \mathrm{H}_{\mathsf{non}}(X \parallel R_0(m,k) \parallel R_1(m,k) \parallel m)$ and $R(m,k) = R_0(m,k) R_1(m,k)^{n(m,k)}$
  end for
end for
Step 4: Signature Generation and Aggregation
for each $m \in \{0,1\}^N$ **do**
  for $k \leftarrow 0$ to $K-1$ **do**
    Both compute: $c(m,k) = \mathrm{H}_{\mathsf{sig}}(X \parallel R(m,k) \parallel m)$
    $\mathcal{P}$: $s_P(m,k) = r_{P,0}(m,k) + n(m,k) r_{P,1}(m,k) + c(m,k) a_P x_P \pmod p$
    $\mathcal{V}$: $s_V(m,k) = r_{V,0}(m,k) + n(m,k) r_{V,1}(m,k) + c(m,k) a_V x_V \pmod p$
    $\mathcal{V}$ sends $s_V(m,k)$ to $\mathcal{P}$
    $\mathcal{P}$ computes: $s(m,k) = s_P(m,k) + s_V(m,k) \pmod p$
    $\mathcal{P}$ verifies: $g^{s(m,k)} \stackrel{?}{=} R(m,k) X^{c(m,k)}$
  end for
end for
Step 5: Hash Transmission and Batch Verification
for each $m \in \{0,1\}^N$ **do**
  for $k \leftarrow 0$ to $K-1$ **do** $\mathcal{P}$ computes: $\mathsf{hs}(m,k) = \mathrm{H}_{\mathsf{sig}}(s(m,k))$
  end for
  $\mathcal{P}$ computes: $H(m) = \mathrm{H}_{\mathsf{sig}}(s(m,0) \parallel s(m,1) \parallel \cdots \parallel s(m,K-1))$
  $\mathcal{P}$ sends all $\mathsf{hs}(m,k)$ and $H(m)$ to $\mathcal{V}$
end for
for each $m \in \{0,1\}^N$ **do**
  for $k \leftarrow 0$ to $K-1$ **do** both compute: $l(m,k) = \mathrm{H}_{\mathsf{sig}}(H(m) \parallel k)$
  end for
  $\mathcal{P}$ sends $s'(m) = \sum_{k=0}^{K-1} l(m,k) s(m,k) \pmod p$
  $\mathcal{V}$ verifies: $g^{s'(m)} \stackrel{?}{=} \prod_{k=0}^{K-1} X^{c(m,k) l(m,k)} \prod_{k=0}^{K-1} R(m,k)^{l(m,k)}$
end for

1. **Setup phase (offline):** During setup, $\mathcal{V}$ verifies that $\mathcal{P}$'s batch hints given by $\{s'(m)\}_{m\in\{0,1\}^N}$ are consistent with the committed signatures. This requires 2^N batch equations, one per possible message m. Under our parameter regime ($N \in \{4, 8\}$), this is at most 256 equations.
2. **Reveal phase (on-chain):** After $\mathcal{P}$ reveals the K signatures

$$\{(R(m(k), k), s(m(k), k))\}_{k=0}^{K-1},$$

verification requires checking only the revealed signatures. If all K revealed messages are distinct, this is K individual Schnorr verifications. If some messages repeat (i.e., $m(k) = m(k')$ for $k \neq k'$), batch verification across those indices reduces the number of expensive group operations.

The on-chain cost scales with K (the number of revealed signatures), not with 2^N (the message space size).

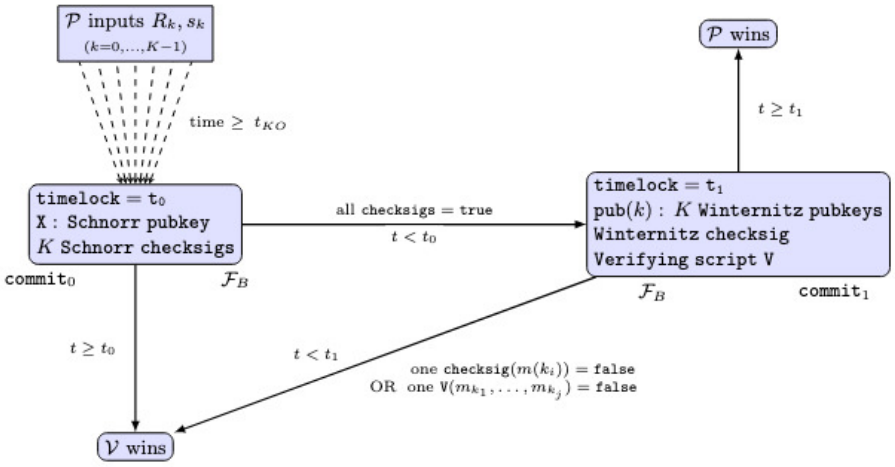

Fig. 1. Correctness resolution mechanism

Inputs and execution The execution of the protocol (Figure 1) after setup follows:

i) When the conditions for kick-off t_{KO} are given (a certain date and time for instance) the first time-lock t_0 starts to run, and $\mathcal{P}$ inputs K signatures in $\texttt{commit}_0$, handled by the trusted party $\mathcal{F}_\mathsf{B}$. The only winning strategy for $\mathcal{P}$ is to input valid signatures

$$R(m(k), k)\ ,\ s(m(k), k), \quad k = 0, 1, \ldots, K-1$$

before timeout t_0, otherwise $\mathcal{V}$ wins.

Algorithm 10 Protocol π_{WISCH} (continued)

Step 6: Winternitz Signature Encryption
for each $m \in \{0,1\}^N$ **do**
  for $k \leftarrow 0$ to $K-1$ **do**
    for $i \leftarrow 0$ to $\ell - 1$ **do** $\mathcal{P}$: $w_i(m,k) = \mathrm{H}_{\mathsf{win}}{}^{b_i(m)}(\mathsf{priv}_i(k))$
    end for
    $\mathcal{P}$: $W(m,k) = (w_0(m,k) \parallel w_1(m,k) \parallel \cdots \parallel w_{\ell-1}(m,k))$
    $\mathcal{P}$: $k_{\mathsf{enc}}(m,k) = \mathsf{HKDF}(s(m,k), m \parallel k)$
    $\mathcal{P}$: $E(m,k) := \mathsf{Enc}^{\mathsf{ad}(sid,k,\mathsf{pub}(k))}_{k_{\mathsf{enc}}(m,k)}\big(W(m,k)\big)$
    $\mathcal{P}$ sends $E(m,k)$ to $\mathcal{V}$
  end for
end for
Step 7: Zero-Knowledge Proof
$\mathcal{P}$ provides ZKP proving:
  (a) $\forall m,k : \mathsf{hs}(m,k) = \mathrm{H}_{\mathsf{sig}}(s(m,k))$
  (b) $\forall m : H(m) = \mathrm{H}_{\mathsf{sig}}(s(m,0) \parallel \cdots \parallel s(m,K-1))$
  (c) $\forall m : s'(m) = \sum_{k=0}^{K-1} l(m,k)s(m,k)$
  (d) $\forall m,k,i : \mathsf{pub}_i(k) = \mathrm{H}_{\mathsf{win}}{}^{2^w-1-b_i(m)}(w_i(m,k))$
  (e) $\forall m,k : E(m,k) := \mathsf{Enc}^{\mathsf{ad}(sid,k,\mathsf{pub}(k))}_{k_{\mathsf{enc}}(m,k)}\big(W(m,k)\big)$
Step 8: Commitments Creation
Create $\mathtt{commit}_0$: aggregated key X, K checksigs, timeout t_0
Create $\mathtt{commit}_1$: Winternitz public keys $\{\mathsf{pub}(k)\}_{k=0}^{K-1}$, verification script V, timeout t_1
Both parties submit to $\mathcal{F}_{\mathsf{B}}$
Commitment Phase (after t_{KO}):
$\mathcal{P}$ selects inputs $\{m(k)\}_{k=0}^{K-1}$ where $m(k) \in \{0,1\}^N$
for $k \leftarrow 0$ to $K-1$ **do** $\mathcal{P}$ submits $(R(m(k),k), s(m(k),k))$ to $\mathtt{commit}_0$ hold by $\mathcal{F}_B$
end for
if $\mathcal{F}_B$ checksigs all K signatures in $\mathtt{commit}_0$ before t_0 **then** go to line 30
else $\mathcal{V}$ wins (timeout)
end if
Revelation and Verification Phase:
for $k \leftarrow 0$ to $K-1$ **do**
  $\mathcal{V}$ determines $m(k)$ from $\mathcal{F}_{\mathsf{B}}$ (unique by $R(m(k),k)$)
  $\mathcal{V}$: $k_{\mathsf{enc}}(m(k),k) = \mathsf{HKDF}(s(m(k),k), m(k) \parallel k)$
  $\mathcal{V}$ decrypts: $W(m(k),k) \leftarrow \mathsf{Dec}^{\mathsf{ad}(sid,k,\mathsf{pub}(k))}_{k_{\mathsf{enc}}(m(k),k)}\big(E(m(k),k)\big)$
  if $W(m(k),k) = \perp$ **then reject**
  end if
  $\mathcal{V}$ verifies: $\forall i : \mathrm{H}_{\mathsf{win}}{}^{2^w-1-b_i(m(k))}(w_i(m(k),k)) \stackrel{?}{=} \mathsf{pub}_i(k)$
end for
$\mathcal{V}$ verifies: $V(\{m(k)\}_{k=0}^{K-1}) \stackrel{?}{=} 1$
Challenge/Resolution Phase:
if $\exists k$: Winternitz verification fails OR $V(\{m(k)\}) = 0$ **then** $\mathcal{V}$ submits challenge to $\mathcal{F}_B$ ($\mathtt{commit}_1$) before time-out t_1
  $\mathcal{F}_B$ verifies $\mathcal{V}$'s claim: she wins
else After t_1: $\mathcal{P}$ wins
end if

Ensure: $\mathcal{P}$ wins if all inputs are valid; $\mathcal{V}$ wins if cheating is detected

ii) Examining these inputs submitted to $\mathcal{F}_{\mathsf{B}}$ by $\mathcal{P}$, $\mathcal{V}$ (who knows the common nonces $R(m(k), k)$ and these are unique for each m, k), learns all the $m(k)$. Using $k_{\mathsf{enc}}(m, k)$ to decrypt $E(m(k), k)$ she also learns $\mathcal{P}$ Winternitz signature $W(m(k), k)$ signed with $\mathsf{priv}(k)$, for all k.

iii) $\mathcal{V}$ verifies that the $W(m(k), k)$ are the signatures of $m(k)$ with $\mathsf{priv}(k)$, by computing

$$\mathrm{H}_{\mathsf{win}}{}^{15-b_i(m(k))}(w_i(m,k)), \quad i \in \{0,\ldots,\ell-1\},\, k \in \{0,\ldots,K-1\}$$

If one of these fails, she gives it to Bob (inputs it in $\mathtt{commit}_1$) who confirms failure before timeout t_1, so she wins.

iv) $\mathcal{V}$ also verifies that the inputs $m(k)$ in fact verify the agreed property of the convened program, if she finds that it is not the case, she inputs it in the Verifier V of $\mathtt{commit}_1$ before timeout t_1, and again she wins.

v) After time-out t_1, $\mathcal{P}$ wins, since this proves that his inputs are correct.

C Security proofs

C.1 Proof of Theorem 1

Proof. We prove this result using a sequence of four hybrid arguments. Each hybrid transition is analyzed below, and the final bound follows by adding up the distinguishing advantages. We start by setting the hybrids as follows:

- H_0 (the real protocol): All parties execute π_{WISCH} with real MuSig2 protocol for joint Schnorr signatures, real Winternitz signatures, real ZKP proofs, and direct interaction with $\mathcal{F}_{\mathsf{B}}$.
- H_1: Same as H_0, except MuSig2 is replaced by $\mathcal{F}_{\mathsf{IMS}}$.
- H_2: Same as H_1, except when corrupted $\mathcal{P}$ provides his ZKP, the simulator additionally runs the knowledge extractor to obtain $\mathcal{P}$'s inputs $\{m(k)\}_{k=0}^{K-1}$ and all Winternitz signatures.
- H_3: Same as H_2, except for each (m, k) where $m \neq m(k)$:
  - Replace $E(m, k)$ with $E(m, k) = E_{k_{\mathsf{enc}}(m,k)}(\text{random})$
  - Keep $E(m(k), k)$ for the K actual inputs
- H_4 (the ideal functionality): Parties interact with $\mathcal{F}_{\mathsf{WISCH}}$. Simulators $\mathcal{S}_{\mathcal{P}}$ and $\mathcal{S}_{\mathcal{V}}$ handle cryptographic operations.

Lemma 2. $|\Pr[\mathcal{Z} = 1 \mid H_0] - \Pr[\mathcal{Z} = 1 \mid H_1]| \leq \epsilon_{\mathsf{IMS}}(\lambda)$

Proof. [BDPT25, Theorem 7] states that MuSig2 UC-realizes $\mathcal{F}_{\mathsf{IMS}}$, we can replace MuSig2 with $\mathcal{F}_{\mathsf{IMS}}$ with distinguishing advantage at most $\epsilon_{\mathsf{IMS}}(\lambda)$.

The reduction $\mathcal{R}$ acts as a perfect relay between $\mathcal{Z}$ and its external MuSig2 interface. From the point of view of $\mathcal{Z}$:

1. All Winternitz operations are computed identically in both hybrids (using real WOTS algorithms).
2. All ZKP operations remain unchanged.

Algorithm 11 Reduction $\mathcal{R}$ for $H_0 \to H_1$

Require: Environment $\mathcal{Z}$, External interface (MuSig2 or $\mathcal{F}_{\mathsf{IMS}}$)
Ensure: $\mathcal{Z}$'s output bit
External Interface: $\mathcal{R}$ interacts with either MuSig2 or $\mathcal{F}_{\mathsf{IMS}}$
Internal Simulation: $\mathcal{R}$ simulates WISCH for $\mathcal{Z}$
procedure SETUP
    Receive parameters from $\mathcal{Z}$ and initialize WISCH protocol
    Generate Winternitz key pairs $\{(\mathsf{priv}(k), \mathsf{pub}(k))\}_{k=0}^{K-1}$
    return initialization success
end procedure
procedure BATCHWINTERNITZCREATION
    for $k \leftarrow 0$ to $K-1$ **do**
        for each $m \in \{0,1\}^N$ **do** Compute $W(m,k) = \text{SIGN}(\mathsf{priv}(k), m)$
        end for
    end for
    return $\{W(m,k)\}_{m,k}$
end procedure
procedure MUSIG2OPERATIONS
    Forward all MuSig2 operations to external interface:
      - KeyGen, SignOff, SignOn, Aggregate, Verify
    External interface provides either:
      - Real cryptographic values (if MuSig2)
      - Simulated values (if $\mathcal{F}_{\mathsf{IMS}}$)
    return forwarded responses
end procedure
All other WISCH components proceed identically in both worlds
return $\mathcal{Z}$'s output bit

3. All interactions with $\mathcal{F}_{\mathsf{B}}$ remain unchanged.
4. The only difference is whether MuSig2 operations use real cryptography or ideal functionality responses.

Since $\mathcal{R}$ forwards all MuSig2-related queries and responses without modification, the view of $\mathcal{Z}$ when $\mathcal{R}$ interacts with real MuSig2 is identical to its view in H_0, and the view of $\mathcal{Z}$ when $\mathcal{R}$ interacts with $\mathcal{F}_{\mathsf{IMS}}$ is identical to its view in H_1. Therefore, any advantage $\mathcal{Z}$ has in distinguishing H_0 from H_1 translates directly to the advantage of $\mathcal{R}$ in distinguishing real MuSig2 from $\mathcal{F}_{\mathsf{IMS}}$, which is bounded by $\epsilon_{\mathsf{IMS}}(\lambda)$.

Lemma 3. $|\Pr[\mathcal{Z} = 1 \mid H_1] - \Pr[\mathcal{Z} = 1 \mid H_2]| \leq \epsilon_{\mathsf{SLKE}}(\lambda)$.

Proof. H_1 and H_2 differ only in the simulator's internal state. In H_2, the simulator additionally employs the SLKE extractor to obtain witnesses from $\mathcal{P}$'s proof. By the SLKE property (Definition 3):

- The extractor $\mathfrak{E}$ observes $\mathcal{P}^\star$'s queries to the global random oracle.
- When $\mathcal{P}^\star$ produces an accepting proof, $\mathfrak{E}$ extracts witnesses without rewinding.
- Extraction succeeds with probability at least $1 - \epsilon_{\mathsf{SLKE}}(\lambda)$.
- The simulation is sound even under concurrent sessions due to straight-line extraction.

Algorithm 12 SLKE extraction in H_2

Require: ZKP proof $\pi_{\mathcal{P}}$ from corrupt prover $\mathcal{P}^\star$, Access to $\mathcal{G}_{\mathsf{RO}}$
Ensure: Extracted witnesses $\{m(k), W(m,k), s(m,k)\}$ or rejection

procedure SimulatorInH$_2(\pi_{\mathcal{P}})$
  Upon receiving $\mathcal{P}$'s ZKP proof $\pi_{\mathcal{P}}$: Verify the proof as in H_1
  if proof verifies **then**
    Invoke SLKE extractor $\mathfrak{E}$:
      Monitor $\mathcal{P}^\star$'s queries to $\mathcal{G}_{\mathsf{RO}}$
      Extract from observed oracle queries:
        Committed inputs: $m(0), m(1), \ldots, m(K-1)$
        All $2^N K$ Winternitz signatures $W(m,k)$
        All signing keys $s(m,k)$
      Verify extraction consistency: $(x, w) \in R_{\mathsf{WISCH}}$
    Store extracted values internally
    Continue protocol with same external behaviour as H_1
    return extracted values
  else Reject as in H_1
    return $\perp$
  end if
end procedure

Let `extract` denote successful extraction. When the proof verifies, the SLKE extractor $\mathfrak{E}$ outputs:

- The committed inputs: $m(0), m(1), \ldots, m(K-1)$.
- All Winternitz signatures: $\{W(m,k)\}_{m\in\{0,1\}^N, k\in[0,K-1]}$.
- All Schnorr signatures: $\{s(m,k)\}_{m\in\{0,1\}^N, k\in[0,K-1]}$.

Since the view is identical when extraction succeeds (the extraction is internal to the simulator and does not affect protocol messages):

$$\Pr[\mathcal{Z}=1 \mid H_1, \texttt{extract}] = \Pr[\mathcal{Z}=1 \mid H_2, \texttt{extract}]$$

Therefore:

$$|\Pr[\mathcal{Z}=1 \mid H_1] - \Pr[\mathcal{Z}=1 \mid H_2]| \leq \Pr[\neg\texttt{extract}] \leq \epsilon_{\mathsf{SLKE}}(\lambda)$$

This bound holds even under concurrent protocol executions, as the SLKE extraction operates in a straight line without disrupting other sessions.

Lemma 4. $|\Pr[\mathcal{Z}=1 \mid H_2] - \Pr[\mathcal{Z}=1 \mid H_3]| \leq (2^N-1)K \cdot \epsilon_{\mathsf{ENC}}(\lambda)$

Proof. We proceed by a hybrid argument over all (m,k) pairs where $m \neq m(k)$. For each $k \in \{0,\ldots,K-1\}$, there are exactly 2^N-1 values $m \in \{0,1\}^N \setminus \{m(k)\}$ whose encryptions are replaced. Thus the total number of replacements is exactly $(2^N-1)K$.

Define intermediate hybrids $H_2 = H_{2,0}, H_{2,1}, \ldots, H_{2,J} = H_3$ where we set $J = (2^N-1)K$, and hybrid $H_{2,j}$ replaces the first j such encryptions with encryptions of random strings. For the transition $H_{2,j} \to H_{2,j+1}$, let $(m^\star, k^\star)$ be the $(j+1)$-th pair with $m^\star \neq m(k^\star)$. We construct an IND-CPA adversary $\mathcal{B}_{j\to j+1}$:

Algorithm 13 IND-CPA adversary $\mathcal{B}_{j\to j+1}$

Require: Hybrid index j, Environment $\mathcal{Z}$
Ensure: Distinguishing bit from $\mathcal{Z}$
Let $(m^\star, k^\star_{\text{idx}})$ be the $(j+1)$-th pair where $m^\star \neq m(k^\star_{\text{idx}})$
$M_0 \leftarrow W(m^\star, k^\star_{\text{idx}})$, $M_1 \xleftarrow{\$} \{0,1\}^{\kappa\cdot\ell}$ ▷ κ is the WOTS output bitlength
procedure KeyGeneration
    Sample fresh pre-key $k^\star_{\mathsf{pre}} \xleftarrow{\$} \{0,1\}^{\kappa_{\mathsf{KDF}}}$
    Program HKDF: $\mathsf{HKDF}\big(s(m^\star, k^\star_{\text{idx}}),\ m^\star \parallel k^\star_{\text{idx}}\big) := k^\star_{\mathsf{pre}}$
    return $k^\star_{\mathsf{pre}}$
end procedure
Send (M_0, M_1) to the IND-CPA challenger using key $k^\star_{\mathsf{pre}}$
Receive $C^\star$; simulate all other encryptions via HKDF as usual
return $\mathcal{Z}$'s output

If $C^\star$ encrypts M_0, then $\mathcal{Z}$ sees hybrid j. If $C^\star$ encrypts M_1, then $\mathcal{Z}$ sees hybrid $j+1$. Thus each step has advantage bounded by $\epsilon_{\mathsf{ENC}}(\lambda)$. We observe that $\mathcal{V}$ only decrypts $E(m(k),k)$ after $\mathcal{P}$ reveals $m(k)$, and these always contain real signatures. In the WISCH protocol, $\mathcal{V}$ only decrypts $E(m(k),k)$ after $\mathcal{P}$

reveals his chosen inputs $\{m(k)\}_{k=0}^{K-1}$ by submitting the corresponding Schnorr signatures. For any (m, k) where $m \neq m(k)$, the encryption $E(m, k)$ is never decrypted because:

1. $\mathcal{P}$ never submits the signature $(R(m, k), s(m, k))$ for $m \neq m(k)$.
2. Without this signature $\mathcal{V}$ cannot compute $k_{\mathsf{enc}}(m, k)$.
3. Without the decryption key, $E(m, k)$ remains unopened.

Summing over exactly $(2^N - 1)K$ hybrid steps gives the stated bound.

Lemma 5. $|\Pr[\mathcal{Z} = 1 \mid H_3] - \Pr[\mathcal{Z} = 1 \mid H_4]| \leq \epsilon_I + \epsilon_L$

Proof. We analyse the transition by showing that both H_3 and H_4 use the same $\mathcal{F}_{\mathsf{B}}$ functionality. The simulator $\mathcal{S}$ in H_4 acts as an interface between $\mathcal{F}_{\mathsf{WISCH}}$ and $\mathcal{F}_{\mathsf{B}}$, making the same calls to $\mathcal{F}_{\mathsf{B}}$ that would occur in the real protocol, using the inputs extracted in H_2. We examine each phase of the protocol and provide explicit simulator behavior. By construction of H_3, the simulator possesses:

1. $\mathcal{P}$'s inputs $\{m(k)\}_{k=0}^{K-1}$ (extracted in H_2).
2. Schnorr signatures $\{s(m, k)\}_{m,k}$ and Winternitz signatures $\{W(m, k)\}_{m,k}$.
3. The verification predicate V and all public parameters.

1. Setup phase simulation: In H_3, parties generate Winternitz keys, execute MuSig2 via $\mathcal{F}_{\mathsf{IMS}}$, and create commitments via $\mathcal{F}_{\mathsf{B}}$. In H_4, the simulator $\mathcal{S}$ interacts with $\mathcal{F}_{\mathsf{B}}$: when $\mathcal{F}_{\mathsf{WISCH}}$ initiates setup, $\mathcal{S}$ calls $\mathcal{F}_{\mathsf{B}}$.CreateCommitment with identical parameters as in H_3; $\mathcal{F}_{\mathsf{B}}$ returns the same commitment IDs it would in real execution; the view is identical unless $\mathcal{F}_{\mathsf{B}}$ fails (probability ϵ_L).

Algorithm 14 Setup simulation for $H_3 \to H_4$

Require: Parameters K, N, n, w, Parties $\mathcal{P}$, $\mathcal{V}$, Access to $\mathcal{F}_{\mathsf{B}}$
Ensure: Simulated setup phase indistinguishable from real

procedure SimulateSetup
  for $k \leftarrow 0$ to $K - 1$ **do**: Generate $(\mathsf{priv}(k), \mathsf{pub}(k))$ as in real protocol
  end for
  MuSig2 execution: (both use $\mathcal{F}_{\mathsf{IMS}}$)
    Forward all MuSig2 operations to $\mathcal{F}_{\mathsf{IMS}}$
  Commitment creation via $\mathcal{F}_{\mathsf{B}}$:
    Invoke $\mathcal{F}_{\mathsf{B}}$.CreateCommitment($\{\mathcal{P}, \mathcal{V}\}$, t_0, $\mathcal{V}$)
    Receive cid_0 from $\mathcal{F}_{\mathsf{B}}$
    Invoke $\mathcal{F}_{\mathsf{B}}$.CreateCommitment($\{\mathcal{P}, \mathcal{V}\}$, t_1, $\mathcal{P}$)
    Receive cid_1 from $\mathcal{F}_{\mathsf{B}}$
  Store commitment identifiers for protocol execution:
    Store $(cid_0, timeout = t_0, parties = \{\mathcal{P}, \mathcal{V}\})$
    Store $(cid_1, timeout = t_1, parties = \{\mathcal{P}, \mathcal{V}\})$
  return commitment identifiers (cid_0, cid_1)
end procedure

The simulation is perfect unless $\mathcal{F}_{\mathrm{B}}$ fails to create commitments, which occurs with probability at most ϵ_L by liveness.

2. Commitment phase simulation: In H_3, $\mathcal{P}$ submits signatures to $\mathcal{F}_{\mathrm{B}}$, which verifies them and updates commitment state. In H_4 the simulator operates as follows:

Algorithm 15 Commitment phase simulation for $H_3 \to H_4$

Require: $\mathcal{P}$'s inputs $\{m(k)\}_{k=0}^{K-1}$, Aggregated key X, Current time, Access to $\mathcal{F}_{\mathsf{B}}$
Ensure: Simulated commitment phase responses

$\mathcal{F}_{\mathsf{WISCH}}$ knows $\mathcal{P}$'s true inputs $\{m(k)\}_{k=0}^{K-1}$
for $k \leftarrow 0$ to $K-1$ **do**
    Query $\mathcal{F}_{\mathsf{IMS}}$ for signature on $m(k)$
    Obtain $(R(m(k),k), s(m(k),k))$
end for
procedure SimulateSignatureSubmission
    When $\mathcal{P}$ submits signatures to $\texttt{commit}_0$:
    for $k \leftarrow 0$ to $K-1$ **do**
        Prepare witness: $w_k = (R(m(k),k), s(m(k),k))$
        Invoke $\mathcal{F}_{\mathsf{B}}$.SubmitCommitment($\mathcal{P}$, cid_0, w_k)
        $\mathcal{F}_{\mathsf{B}}$ executes Schnorr verification script
        Receive verification result from $\mathcal{F}_{\mathsf{B}}$
        Forward $\mathcal{F}_{\mathsf{B}}$'s response to relevant parties
    end for
end procedure
procedure HandleTimeout
    Query $\mathcal{F}_{\mathsf{B}}$.Query($\texttt{time}$) for current time
    if *CurrentTime* $\geq t_0$ and not all K signatures received **then**
        $\mathcal{F}_{\mathsf{B}}$.ResolveCommitment(cid_0, $\mathcal{V}$) ▷ Automatic timeout resolution
        Forward resolution notification to all parties
    end if
end procedure

Since $\mathcal{S}$ knows the correct inputs from extraction, it submits the exact same signatures to $\mathcal{F}_{\mathsf{B}}$ that $\mathcal{P}$ would submit in H_3. The same $\mathcal{F}_{\mathsf{B}}$ functionality performs verification in both worlds, producing identical results unless $\mathcal{F}_{\mathsf{B}}$ violates its integrity specification (probability ϵ_I).

3. Revelation phase simulation: In H_3, $\mathcal{V}$ extracts inputs from $\mathcal{F}_{\mathsf{B}}$ witnesses and decrypts Winternitz signatures. Concerning H_4, this phase requires no special simulation because:

   Moreover, $\mathcal{V}$ retrieves witnesses from $\mathcal{F}_{\mathsf{B}}$ identically in both worlds via Query operations. The stored witnesses are identical because $\mathcal{S}$ submitted the same values that $\mathcal{P}$ would have submitted.

4. Challenge and resolution phase simulation: in this phase one has H_3 where $\mathcal{V}$ may submit challenges to $\mathcal{F}_{\mathsf{B}}$, which executes verification scripts. Concerning H_4, the simulator operates as follows:

Algorithm 16 Revelation phase simulation for $H_3 \to H_4$

Require: Revealed inputs $\{m(k)\}_{k=0}^{K-1}$, Schnorr signatures $\{s(m(k), k)\}$, Access to $\mathcal{F}_{\mathsf{B}}$
Ensure: Simulated revelation for honest $\mathcal{V}$

procedure SIMULATEREVELATION
  $\mathcal{F}_{\mathsf{WISCH}}$ sends revealed inputs to honest $\mathcal{V}$
  Send $(\texttt{revealed}, \{m(k)\}_{k=0}^{K-1})$ to $\mathcal{V}$
  Enable $\mathcal{V}$ to query $\mathcal{F}_{\mathsf{B}}$ for witnesses:
  for $k \leftarrow 0$ to $K-1$ **do**
    $\mathcal{V}$ queries: $\mathcal{F}_{\mathsf{B}}$.Query(`witnesses`, cid_0)
    $\mathcal{F}_{\mathsf{B}}$ returns $(R(m(k), k), s(m(k), k))$ from stored witnesses
    $\mathcal{V}$ determines $m(k)$ from unique nonce $R(m(k), k)$
  end for
  $\mathcal{V}$'s local computation: (identical in both hybrids)
  for $k \leftarrow 0$ to $K-1$ **do**
    Compute $k_{\mathsf{enc}}(m(k), k) = \mathsf{HKDF}(s(m(k), k), m(k) \parallel k)$
    Decrypt $E(m(k), k)$ to obtain $W(m(k), k)$
    Verify Winternitz signatures locally
  end for
  return verification results
end procedure

The simulation differs from H_3 if $\mathcal{F}_{\mathrm{B}}$ incorrectly evaluates the verification script (probability bounded by ϵ_I), or if $\mathcal{F}_{\mathrm{B}}$ fails to process a valid challenge before timeout (with probability bounded by ϵ_L).
We observe that $\mathcal{F}_{\mathsf{B}}$ is not simulated but shared between both worlds. The simulator, having perfect information from the extraction in H_2, makes identical calls to $\mathcal{F}_{\mathsf{B}}$ that would occur in the real protocol. The environment's view differs only when either $\mathcal{F}_{\mathsf{B}}$ violates integrity: incorrect script execution (probability ϵ_I), or $\mathcal{F}_{\mathsf{B}}$ violates liveness: failure to process operations (probability ϵ_L).

Therefore: $|\Pr[\mathcal{Z} = 1 \mid H_3] - \Pr[\mathcal{Z} = 1 \mid H_4]| \leq \epsilon_I + \epsilon_L$.

Combining all hybrid transitions with the triangle inequality:

$$|\Pr[\mathcal{Z} = 1 \mid H_0] - \Pr[\mathcal{Z} = 1 \mid H_4]| \leq \sum_{i=0}^{3} |\Pr[\mathcal{Z} = 1 \mid H_i] - \Pr[\mathcal{Z} = 1 \mid H_{i+1}]|$$
$$\leq \epsilon_{\mathsf{IMS}}(\lambda) + \epsilon_{\mathsf{SLKE}}(\lambda) + (2^N - 1)K \cdot \epsilon_{\mathsf{ENC}}(\lambda) + \epsilon_I + \epsilon_L$$

This ends the proof of Theorem 1.

C.2 Proof of Theorem 2

Proof. We prove the lemma using a sequence of five hybrid arguments. Each hybrid transition is analyzed below, and the final bound follows by adding up the distinguishing advantages. We start by setting the hybrids as follows:

Algorithm 17 Challenge phase simulation for $H_3 \to H_4$

Require: Inputs $\{m(k)\}_{k=0}^{K-1}$, Winternitz signatures, Verification predicate V, Access to $\mathcal{F}_{\mathsf{B}}$
Ensure: Simulated challenge resolution
$\mathcal{F}_{\mathsf{WISCH}}$ knows true verification outcomes
for $k \leftarrow 0$ to $K-1$ **do**: $WOTS\_valid[k] \leftarrow \texttt{WVerify}(m(k), W(m(k),k), \mathsf{pub}(k))$
end for
$program\_valid \leftarrow V(\{m(k)\}_{k=0}^{K-1})$
procedure SimulateChallenge($\mathcal{V}$, *type*, *witness*)
    When $\mathcal{V}$ submits challenge to $\texttt{commit}_1$: Query current time: $CurrentTime \leftarrow \mathcal{F}_{\mathsf{B}}$.Query(`time`)
    if $CurrentTime < t_1$ **then**
        if $type = \texttt{WOTS}$ and $witness = (k, m(k), \sigma_k)$ **then**
            Prepare challenge: $c = (inputs = \sigma_k, outputs = \bot, witnesses = (k, m(k)))$
            Invoke $\mathcal{F}_{\mathsf{B}}$.SubmitCommitment($\mathcal{V}$, cid_1, c)
            $\mathcal{F}_{\mathsf{B}}$ executes WOTS verification script
            $result \leftarrow \mathcal{F}_{\mathsf{B}}$.ExecuteScript($cid_1$, `winternitz`)
            if not *result* **then** $\mathcal{F}_{\mathsf{B}}$.ResolveCommitment(cid_1, $\mathcal{V}$)
            end if
        else if $type = \texttt{program}$ and $witness = \{m(k)\}_{k=0}^{K-1}$ **then**
            Prepare challenge: $c = (inputs = \{m(k)\}, outputs = \bot, witnesses = \bot)$
            Invoke $\mathcal{F}_{\mathsf{B}}$.SubmitCommitment($\mathcal{V}$, cid_1, c)
            $\mathcal{F}_{\mathsf{B}}$ executes program verification V
            $result \leftarrow \mathcal{F}_{\mathsf{B}}$.ExecuteScript($cid_1$, `program`)
            if not *result* **then** $\mathcal{F}_{\mathsf{B}}$.ResolveCommitment(cid_1, $\mathcal{V}$)
            end if
        end if
    end if
end procedure
procedure HandleFinalTimeout
    Query: $CurrentTime \leftarrow \mathcal{F}_{\mathsf{B}}$.Query(`time`)
    if $CurrentTime \geq t_1$ **then**
        Query: $resolved \leftarrow \mathcal{F}_{\mathsf{B}}$.Query(`resolved`, cid_1)
        if $resolved = \bot$ **then**
            $\mathcal{F}_{\mathsf{B}}$.ResolveCommitment(cid_1, $\mathcal{P}$)
        end if
    end if
end procedure

- G_0 (the real protocol): Honest party $\mathcal{P}$ executes π_{WISCH} with corrupt $\mathcal{V}^\star$, real MuSig2 protocol for joint Schnorr signatures, real ZK proofs, and direct interaction with $\mathcal{F}_{\mathsf{B}}$.
- G_1: Same as G_0, except MuSig2 is replaced by $\mathcal{F}_{\mathsf{IMS}}$.
- G_2: Same as G_1, except all ZK proofs from honest $\mathcal{P}$ to corrupt $\mathcal{V}^\star$ are replaced with simulated proofs.
- G_3: Same as G_2, except for each (m, k), the simulator encrypts the real $W(m, k)$ under uniformly random keys $k_{\mathsf{pre}}(m, k)$ instead of $k_{\mathsf{enc}}(m, k)$.
- G_4 (the ideal functionality): At reveal, the simulator programs HKDF so that $\mathsf{HKDF}(s(m(k), k), m(k)\|k) = k_{\mathsf{pre}}(m(k), k)$ for the revealed inputs.

Lemma 6. $|\Pr[\mathcal{Z} = 1 \mid G_0] - \Pr[\mathcal{Z} = 1 \mid G_1]| \leq \epsilon_{\mathsf{IMS}}(\lambda)$

Proof. By [BDPT25, Theorem 7], MuSig2 UC-realizes $\mathcal{F}_{\mathsf{IMS}}$. We can replace MuSig2 with $\mathcal{F}_{\mathsf{IMS}}$ with distinguishing advantage at most $\epsilon_{\mathsf{IMS}}(\lambda)$. The reduction acts as a perfect relay between $\mathcal{Z}$ and its external MuSig2 interface. All other protocol components remain unchanged.

Lemma 7. $|\Pr[\mathcal{Z} = 1 \mid G_1] - \Pr[\mathcal{Z} = 1 \mid G_2]| \leq \epsilon_{\mathsf{ZK}}(\lambda)$

Proof. G_1 and G_2 differ only in how ZK proofs are generated. In G_2, the simulator creates simulated proofs without using witnesses. By the zero-knowledge property in the GRO model with domain-separated oracle calls, the distribution of simulated proofs is computationally indistinguishable from real proofs with advantage bounded by $\epsilon_{\mathsf{ZK}}(\lambda)$.

Lemma 8. $|\Pr[\mathcal{Z} = 1 \mid G_2] - \Pr[\mathcal{Z} = 1 \mid G_3]| = 0$

Proof. We show that the distributions of ciphertexts in G_2 and G_3 are identical from $\mathcal{V}^\star$'s view. In G_2, the encryption key for (m, k) is

$$k_{\mathsf{enc}}(m, k) = \mathsf{HKDF}(s(m, k), m\|k).$$

In G_3, the encryption key is $k_{\mathsf{pre}}(m, k) \xleftarrow{\$} \{0,1\}^{\kappa_{\mathsf{KDF}}}$.

We argue that $k_{\mathsf{enc}}(m, k)$ is uniformly distributed over $\{0,1\}^{\kappa_{\mathsf{KDF}}}$ from $\mathcal{V}^\star$'s view:

1. **Schnorr scalar secrecy:** The aggregated signature scalar is given by

$$s(m, k) = s_P(m, k) + s_V(m, k) \pmod p.$$

The honest prover's share is:

$$s_P(m, k) = r_{P,0}(m, k) + \eta(m, k) \cdot r_{P,1}(m, k) + c(m, k) \cdot a_P \cdot x_P \pmod p$$

where $r_{P,0}(m, k), r_{P,1}(m, k) \xleftarrow{\$} \mathbb{Z}_p^\star$ are freshly sampled by honest $\mathcal{P}$ and never revealed to $\mathcal{V}^\star$ before the reveal phase. Thus $s_P(m, k)$, and hence $s(m, k)$, is uniformly distributed over $\mathbb{Z}_p$ from $\mathcal{V}^\star$'s view.

2. **HKDF in the GRO model:** We model HKDF as a random oracle with domain separation. For any input (s, y) not previously queried, $\mathsf{HKDF}(s, y)$ returns a uniformly random κ_{KDF}-bit string.
3. **Pre-reveal query independence:** Before reveal, $\mathcal{V}^\star$ has not observed $s(m, k)$ for any (m, k). Since $s(m, k)$ is uniform over $\mathbb{Z}_p$, the probability that $\mathcal{V}^\star$ queries HKDF on exactly $(s(m, k), m\|k)$ is negligible (this is formalized in the $G_3 \to G_4$ transition via the freshness bound).

Therefore, from $\mathcal{V}^\star$'s view before reveal, $k_{\mathsf{enc}}(m, k) = \mathsf{HKDF}(s(m, k), m\|k)$ is a uniformly random κ_{KDF}-bit string, identically distributed to $k_{\mathsf{pre}}(m, k)$. Since encryption is deterministic given the key and plaintext, the ciphertext distributions are identical.

Lemma 9. $|\Pr[\mathcal{Z} = 1 \mid G_3] - \Pr[\mathcal{Z} = 1 \mid G_4]| \leq q_H 2^{-\ell_s}$.

Proof. Let $\mathsf{EarlyQuery}$ be the event from Definition 5. By Lemma 1, we have: $\Pr[\mathsf{EarlyQuery}] \leq q_H 2^{-\ell_s}$.

Conditioned on $\neg\mathsf{EarlyQuery}$, at reveal time the simulator programs

$$\mathsf{HKDF}(s(m(k), k), m(k)\|k) := k_{\mathsf{pre}}(m(k), k)$$

for each $k \in [0, K-1]$. This programming is undetectable since these are fresh queries in the GRO model. With this programming, $\mathcal{V}^\star$ computes the decryption keys that recover the real $W(m(k), k)$ values, making verification identical to G_3. Therefore:

$$|\Pr[\mathcal{Z} = 1 \mid G_3] - \Pr[\mathcal{Z} = 1 \mid G_4]| \leq q_H 2^{-\ell_s}$$

Finally, G_4 corresponds to the ideal execution with $\mathcal{F}_{\mathsf{WISCH}}$ and simulator $\mathcal{S}_{\mathcal{V}}$. The simulator operates as follows:

- Generates Winternitz key pairs and executes MuSig2 via $\mathcal{F}_{\mathsf{IMS}}$.
- For each (m, k), computes real $W(m, k)$ using $\mathsf{priv}(k)$ and encrypts under random keys $k_{\mathsf{pre}}(m, k)$.
- Creates simulated ZK proofs without witnesses.
- Upon receiving revealed inputs from $\mathcal{F}_{\mathsf{WISCH}}$, submits signatures to $\mathcal{F}_{\mathsf{B}}$ and programs HKDF appropriately.

The simulation succeeds except when one of the distinguishing events occurs. Combining all hybrid transitions with the triangle inequality:

$$\begin{aligned}|\Pr[\mathcal{Z} = 1 \mid G_0] - \Pr[\mathcal{Z} = 1 \mid G_4]| &\leq \sum_{i=0}^{3} |\Pr[\mathcal{Z} = 1 \mid G_i] - \Pr[\mathcal{Z} = 1 \mid G_{i+1}]| \\ &\leq \epsilon_{\mathsf{IMS}}(\lambda) + \epsilon_{\mathsf{ZK}}(\lambda) + \epsilon_I + \epsilon_L + q_H 2^{-\ell_s}\end{aligned}$$

This completes the proof of Theorem 2.

C.3 Proof of Theorem 3

Proof. **Completeness:** When both parties are honest and $\mathcal{P}$ has valid inputs $\{m(k)\}_{k=0}^{K-1}$ with $V(\{m(k)\}) = 1$:

1. Setup phase success:
   - By MuSig2 correctness, all joint Schnorr signatures $s(m(k), k)$ are correctly generated with probability $1 - \epsilon_{\mathsf{S}}(\lambda)$.
   - By perfect completeness of the ZK proof system, an honest $\mathcal{P}$ with a valid witness always produces an accepting proof.
   - Commitments $\texttt{commit}_0$ and $\texttt{commit}_1$ are created with probability $1-\epsilon_L$.
2. Execution phase: After time t_{KO}:
   - $\mathcal{P}$ submits valid $(R(m(k), k), s(m(k), k))$ for $k = 0, \ldots, K-1$ to $\texttt{commit}_0$.
   - All K checksigs pass since signatures are valid.
   - The commitment $\texttt{commit}_1$ is created before timeout t_0.
3. Verification phase:
   - $\mathcal{V}$ extracts $m(k)$ from $\mathcal{F}_{\mathsf{B}}$ witnesses (deterministic given unique nonces).
   - $\mathcal{V}$ decrypts $E(m(k), k)$ to obtain $W(m(k), k)$.
   - For each k: $\texttt{WVerify}(m(k), W(m(k), k), \texttt{pub}_k) = 1$ with probability $1 - \epsilon_{\mathsf{W}}(\lambda)$ (WOTS correctness).
   - $V(\{m(k)\}) = 1$ by assumption.
   - Since all checks pass, $\mathcal{V}$ does not challenge.
4. After timeout t_1, the commitment $\texttt{commit}_1$ resolves for $\mathcal{P}$.

The probability of failure is bounded by the union of failure events:

$$\epsilon_{\mathsf{comp}} = \epsilon_{\mathsf{S}}(\lambda) + \epsilon_{\mathsf{W}}(\lambda) + \epsilon_L$$

Soundness: Let $\mathcal{A}$ be a PPT adversary controlling $\mathcal{P}$. If $\mathcal{P}$'s inputs are invalid (there exists k such that $\texttt{WVerify}(m(k), \sigma_k, \texttt{pub}_k) \neq 1$ or $V(\{m(k)\}) = 0$), then $\texttt{commit}_1$ cannot resolve in favour of $\mathcal{A}$ except with negligible probability; indeed, the following two situations arise:

Case 1: $\mathcal{A}$ fails to provide valid Schnorr signatures.

- Nonce uniqueness: Each aggregated nonce

  $$R(m, k) = R_0(m, k) \cdot R_1(m, k)^{\eta(m,k)}$$

  is determined by independent random choices from both parties. For honest $\mathcal{V}$, the nonces $r_{V,0}(m, k), r_{V,1}(m, k)$ are sampled uniformly from $\mathbb{Z}_p^\star$ for each (m, k). The probability that any two distinct pairs $(m, k) \neq (m', k')$ yield $R(m, k) = R(m', k')$ is at most $1/p$ per pair. By a union bound over all $\binom{2^N K}{2}$ pairs:

  $$\epsilon_{\mathsf{nonce}}(\lambda) \leq \binom{2^N K}{2} \cdot \frac{1}{p} < \frac{(2^N K)^2}{2p}.$$

  Under our parameter regime ($N \in \{4, 8\}$, $K = \text{poly}(\lambda)$, and $p \approx 2^{256}$), this is negligible.

- Conditioned on nonce uniqueness, the binding $c(m,k) = \mathrm{H}_{\mathsf{sig}}(X \parallel R(m,k) \parallel m)$ ensures that each valid signature $(R(m,k), s(m,k))$ corresponds to exactly one message m.
- By EUF-CMA security of MuSig2 (UC-realized via $\mathcal{F}_{\mathsf{IMS}}$), $\mathcal{A}$ cannot forge a signature for any (m,k) not jointly signed during setup, except with probability $\epsilon_{\mathsf{S}}(\lambda)$.
- If $\mathcal{A}$ fails to provide all K valid signatures by timeout t_0, $\texttt{commit}_0$ resolves for $\mathcal{V}$ (Figure 1).

Case 2: $\mathcal{A}$ provides valid Schnorr signatures but cheats on Winternitz or program verification. Assume $\mathcal{A}$ provides $(R(m(k),k), s(m(k),k))$ for some inputs $\{m(k)\}_{k=0}^{K-1}$.

- Input extraction: by nonce uniqueness, $\mathcal{V}$ uniquely determines $m(k)$ from each $(R(m(k),k), s(m(k),k))$ pair.
- Winternitz decryption: $\mathcal{V}$ computes $k_{\mathsf{enc}}(m(k),k)$ and decrypts $E(m(k),k)$ to obtain σ_k.
- ZKP guarantee: by knowledge soundness of the ZK system, the probability that an accepting proof attests a false statement is at most $\epsilon_{\mathsf{SLKE}}(\lambda)$.
- Invalid input detection: if either there exists a value k such that

$$\texttt{WVerify}(m(k), \sigma_k, \texttt{pub}_k) \neq 1, \text{ or } V(\{m(k)\}) = 0$$

  then $\mathcal{V}$ detects the invalidity and challenges before timeout t_1.
- Challenge success:
  - For Winternitz forgery: by EUF-OT-CMA security, $\mathcal{A}$ cannot produce valid σ_k for $m(k)$ without knowing $\texttt{priv}_k$, except with probability $\epsilon_{\mathsf{W}}(\lambda)$.
  - For program violation: the verification V is deterministic, so $\mathcal{V}$'s challenge succeeds with certainty.
- Commitment resolution: on successful challenge, $\texttt{commit}_1$ resolves in favour of $\mathcal{V}$.

The probability of $\texttt{commit}_1$ resolving in favour of $\mathcal{A}$ despite invalid inputs is bounded by the union of failure events:

$$\epsilon_{\mathsf{sound}}(\lambda) = \epsilon_{\mathsf{S}}(\lambda) + \epsilon_{\mathsf{SLKE}}(\lambda) + \epsilon_{\mathsf{W}}(\lambda) + \epsilon_{\mathsf{nonce}}(\lambda) + \epsilon_I.$$

D Performance analysis details

We analyse the commitment size in $\mathcal{F}_{\mathsf{B}}$ of WISCH compared to alternative signature schemes. Our analysis focuses on the size in bytes required for K signatures, where K represents the number of challenge rounds in the protocol.

WISCH size analysis WISCH makes use of $\mathcal{F}_{\mathsf{B}}$ script separator, which allows multiple Schnorr signature checks for the same public key X within a single script. This enables to amortize the cost of storing the public key across all

signatures. Indeed: for K Schnorr signatures using the same public key X, the WISCH script size is: $\text{Size}_{\mathsf{WISCH}} = 33 + K \times 68$ bytes where 33 bytes store the compressed public key X, and each signature check requires 68 bytes. This follows from the analysis below:

For the WISCH protocol, we use Schnorr signatures with an elliptic curve E over $\mathbb{Z}_p$ where $p < 2^\lambda$:

1. Public key size: an aggregated Schnorr public key X is a point on the curve. Since we need to store $X_x \in \mathbb{Z}_p$(32 bytes) plus a parity byte to recover X_y: $\text{Size}(X) = 32 + 1 = 33$ bytes.
2. Size per signature check: a Schnorr signature (R, s) consists of 32 bytes from $R_x \in \mathbb{Z}_p$, 32 bytes from $s \in \mathbb{Z}_p$, an 1 byte from a parity byte for R_y. Thus each signature occupies $32 + 32 + 1 = 65$ bytes. In the WISCH script, each signature check requires the signature $(R(m(k), k), s(m(k), k))$: 65 bytes, and script opcodes for verification: 3 bytes. This gives $65 + 3 = 68$ bytes per signature check.
3. Total script size: Multiple Schnorr signature checks for the same public key X can be included in a single $\mathcal{F}_\mathsf{B}$ script separator. Therefore, for K signature checks: $\text{Size}_{\mathsf{WISCH}} = \underbrace{33}_{\text{public key } X} + \underbrace{K \times 68}_{\text{K signaturechecks}}$ bytes

For large K this is approximately $K \times 68$ bytes since the 33-byte public key becomes negligible, achieving near-optimal scaling.

Comparison with hash-based signatures Below follows a comparison of WISCH instantiated by SHA256 against two standard hash-based signature schemes:

1. Lamport signatures:
   (a) Each signature requires a public key consisting of 512 hash values (for signing 0 and 1 bits).
   (b) Each signature reveals 256 preimages.
   (c) Total size for K signatures:

$$\text{Size}_{\mathsf{Lamport}} = K \times \underbrace{(2+1)}_{\text{pub+sig}} \times 256 \times 32 = K \times 24576 \text{ bytes.}$$

2. Winternitz one-time signatures:
   (a) Uses parameter w to trade computation for size.
   (b) Messages of N bits require $\ell = M + C$ chunks where: $M = \lceil N/w \rceil$ (message chunks).
   (c) Total size for K signatures: $\text{Size}_{\mathsf{WOTS}} = K \times 2 \times \ell \times 32$ bytes.

N	w	ℓ	WOTS size	WISCH size
16	4	6	$K \times 384$ bytes	$K \times 68$ bytes
24	4	8	$K \times 512$ bytes	$K \times 68$ bytes
32	4	10	$K \times 640$ bytes	$K \times 68$ bytes

Table 2. On-chain size of K WOTS

Efficiency gains The table below compares WISCH against optimized WOTS configurations:

From the table above we observe that:

1. Compared to Lamport: WISCH is 22-45× smaller approximately.
2. Compared to optimized WOTS, and depending on parameters: WISCH is 6-9× smaller approximately.
3. WISCH uses only 15% of the on-chain space required by WOTS.

These improvements translate directly to reduced commitment operations and make previously impractical applications viable.

Remark 12. WISCH treats $W(m,k)$ as an arbitrary verifiable artifact bound to a public key $\mathsf{pub}(k)$, with a deterministic predicate $\mathsf{Verify}(\mathsf{pub}(k), m, W)$ which either accepts or rejects its input. We instantiate W with Winternitz OTS, but the protocol and proofs go through unchanged if we replace it by any one-time signature with deterministic verification, such as Lamport signatures, provided the ZK relation asserts that, for all m and k, $E(m,k)$ encrypts a value $W(m,k)$ such that it gets successfully verified. Similarly, the verification can be realized by evaluating a public circuit C_{Verify} (using garbled circuits). In the forthcoming security analysis, one may observe that the reductions do not depend on the specific instantiation of Verify.

E Mixed-role corruption analysis

We consider the mixed-role corruption model. A PPT adversary may statically corrupt either the prover or the verifier at the start of the execution, and then interleave arbitrarily many concurrent sessions. All other aspects of the execution model remain unchanged with respect to the single-role corruption model.

Definition 23 (Global bad event for simulation soundness). *In a multi-session execution of WISCH, let* $\mathsf{Bad}_{\mathsf{SS}}$ *denote the event that the adversary* $\mathcal{A}$*—after observing simulated proofs produced by the simulator in sessions where* $\mathcal{A}$ *plays the verifier role—outputs an accepting proof* $\pi^\star$ *for a statement* $x^\star$ *such that* $x^\star \notin \mathcal{L}_{R_{\mathsf{WISCH}}} := \{x : \exists w, (x,w) \in R_{\mathsf{WISCH}}\}$ *in some session where* $\mathcal{A}$ *plays the prover role. By Definition 2:*

$$\Pr[\mathsf{Bad}_{\mathsf{SS}}] \leq \epsilon_{\mathsf{SS}}(\lambda).$$

Remark 13. Throughout, the public input x of each proof includes at least the session identifier *sid*, the role (prover/verifier), and the protocol's public values for that session. This ensures cross-session/role separation of statements.

Theorem 4. *Under the assumptions of Theorem 1 and assuming simulation soundness, for any static mixed-role PPT adversary and PPT environment,*

$$\epsilon(\lambda) = \epsilon_{\mathsf{IMS}}(\lambda) + \epsilon_{\mathsf{SLKE}}(\lambda) + (2^N - 1)K \cdot \epsilon_{\mathsf{ENC}}(\lambda) + q_H 2^{-\ell_s} + \epsilon_I + \epsilon_L + \epsilon_{\mathsf{SS}}(\lambda).$$

Proof (Sketch of the proof). We run the same hybrid sequence as in Theorem 1. The new aspect under mixed-role corruption is that the adversary may see simulated proofs (as a corrupt verifier in some sessions) while attempting to produce accepting proofs as a corrupt prover in other sessions. Introduce the global event $\mathrm{Bad}_{\mathsf{SS}}$ and condition all hybrids on $\neg\mathrm{Bad}_{\mathsf{SS}}$. Under this conditioning, no hybrid change differs from the single-role proof: $H_1 \to H_2$ still uses straight-line extraction (SLKE); the encryption hop is unchanged; ledger terms and HKDF freshness are unchanged. Finally, we only need to add $\Pr[\mathrm{Bad}_{\mathsf{SS}}] \le \epsilon_{\mathsf{SS}}(\lambda)$ by a union bound with the distinguishing advantages.

E.1 Security against a corrupt verifier

Theorem 5. *Let $\epsilon_{\mathsf{ZK}}(\lambda)$ be the ZK indistinguishability advantage used in the corrupt-verifier hybrids and $q_H 2^{-\ell_s}$ the HKDF freshness bound. Under Definition 2, for any PPT adversary controlling verifiers and any PPT environment,*

$$\left|\Pr[\mathcal{Z}^{\pi_{\mathsf{WISCH}},\mathcal{A},\mathcal{F}_{\mathsf{B}}} = 1] - \Pr[\mathcal{Z}^{\mathcal{F}_{\mathsf{WISCH}},\mathcal{S}_{\mathcal{V}}} = 1]\right| \;\le\; \epsilon(\lambda),$$

where $\epsilon(\lambda) = \epsilon_{\mathsf{IMS}}(\lambda) + \epsilon_{\mathsf{ZK}}(\lambda) + q_H 2^{-\ell_s} + \epsilon_I + \epsilon_L + \epsilon_{\mathsf{SS}}(\lambda)$.

Proof (Sketch of the proof). The reasoning here is similar to that in Theorem 4: we use the same hybrids $G_0 \to G_4$ from the main result. Condition on $\neg\mathrm{Bad}_{\mathsf{SS}}$: this prevents simulated proofs observed as a verifier in other sessions from enabling a forgery in any prover session. We finish by adding $\Pr[\mathrm{Bad}_{\mathrm{SS}}] \le \epsilon_{\mathsf{SS}}(\lambda)$ to the end; all intermediate bounds remain identical.